
MITO

Algorithm Specification and Supporting Documentation

**Changlun Li^{1,2}, Chengju Li³, Liang Bai^{1,2}, Qiping Lin^{1,2},
Tianrui Liu³, Yan Ding^{1,2}, Chunyu Gan³, Zhongyu Huang³**

¹ *Beijing Key Laboratory of Post-Quantum Cryptography*

² *Xingtang Telecommunications Technology Co., LTD.*

³ *East China Normal University*

Contact Email

li-cl15@tsinghua.org.cn

June 28, 2026

Contents

1	Introduction	2
1.1	Motivation	2
1.2	An Algebraic Framework for Provable Code-Based KEMs	2
1.3	Mito: An Efficient Code-Based KEM for Static-Key Scenarios	3
2	Mathematical Background	4
2.1	Notations	4
2.2	Coding Theory	5
2.2.1	Adic Codes	6
2.2.2	Dyadic Codes and Dyadic Algebras	7
2.2.3	Concatenated Reed-Muller and Reed-Solomon Codes	9
3	Algorithm Specification	22
3.1	Variables and Functions	22
3.1.1	Variables	22
3.1.2	Functions	24
3.2	The Mito PKE Scheme	26
3.3	The Mito KEM Scheme	29
3.4	System Parameters and Sizes	32
3.4.1	System Parameter Sets	32
3.4.2	Ciphertext and Key Sizes	33
3.4.3	Performance Benchmark	33
4	Security Analysis	35
4.1	Security Assumptions	35
4.1.1	Cryptographic Definitions	35
4.1.2	Syndrome Decoding Problem	38
4.1.3	HQC Assumption	39
4.1.4	Unbalanced Parameters and Projective Structure	42
4.2	Analysis of the Decoding Failure Rate	44
4.2.1	DFR Estimation for Classical RSRM Concatenated Codes	47
4.2.2	DFR Estimation Based on Distance-Informed Erasure Decoding	47
4.3	Security Proof	48
4.3.1	Known Attacks Against the IND-CPA Security	48
4.3.2	IND-CCA Security of KEM	50
4.3.3	Advantages and Limitations	51

1 Introduction

1.1 Motivation

Traditional code-based public-key cryptosystems, such as the original McEliece scheme [McE78], have long suffered from inherently large public keys. While numerous variants have introduced alternative algebraic structures to compress key lengths (e.g., [Gab05, BBC08, BCGO09, Per12]), most of these modifications failed to provide robust security guarantees and were subsequently broken [OTD10, FOPT10, FOP⁺16].

The emergence of *quasi-cyclic* schemes [AMBD⁺18] introduced a new paradigm. Instead of relying on the secrecy of a specifically hidden code, these schemes decouple the core security assumptions from the underlying error-correcting code. This structural shift naturally yields significantly smaller public keys while allowing greater design flexibility. A prominent realization of this paradigm is HQC [GAMA⁺25], a Hamming-metric quasi-cyclic scheme recently standardized by the **National Institute of Standards and Technology** (NIST) as a **Key Encapsulation Mechanism** (KEM).

Building upon this paradigm, our work explores generalizing the *quasi-cyclic* framework to a *quasi-dyadic* structure. Since both families belong to the broader class of *reproducible codes* [SPB21], they share fundamental structural similarities and were historically explored in parallel for McEliece variants [BBB⁺18, BBC⁺19]. Despite extensive recent research on quasi-dyadic codes [BC18, MK22, MPK25], their direct application in constructing KEMs remains a notable gap. To bridge this gap while preserving security, we must first revisit the theoretical foundations that underpin modern code-based KEMs.

1.2 An Algebraic Framework for Provable Code-Based KEMs

A central contribution of this work is a unified algebraic framework that abstracts and generalizes the security foundations of quasi-cyclic schemes such as HQC. While HQC currently lacks an NP-completeness proof for its underlying hard problems, the cryptographic community's confidence in its security is well-founded: the robustness of HQC stems from extensive historical cryptanalysis of random quasi-cyclic codes, together with its careful instantiation of the polynomial quotient ring $\mathbb{F}_2[X]/\langle X^n + 1 \rangle$.

To justify our transition to a quasi-dyadic setting, we re-examine the encryption and decryption mechanisms of quasi-cyclic schemes from an abstract algebraic perspective. We introduce a generalized class of code-based PKE schemes, termed *fully (projective) regular schemes*, and establish a one-to-one correspondence between these schemes and polynomial quotient rings generated by zero-dimensional ideals. We then abstract the security reduction of HQC, formally characterizing the mathematical conditions that an FRS must satisfy to be *provable*, that is, to admit a decomposition $\mathcal{R} \cong \mathcal{R}_{\text{public}} \times \mathcal{R}_{\text{private}}$ where the weight constraints depend only on the public component and the private component is indecomposable.

This framework reveals that both quasi-cyclic and quasi-dyadic PKE schemes are special instantiations of the same algebraic structure. The quasi-dyadic structure, as a distinct object in algebraic coding theory, admits a particularly concise representation and incurs lower computational overhead for polynomial multiplications. We prove that the dyadic algebra satisfies the provability condition (**Theorem 4.1.2**), thereby establishing the rigorous security foundation for quasi-dyadic KEMs. Motivated by these guarantees, we propose the *first* KEM natively built on a quasi-dyadic structure.

1.3 Mito: An Efficient Code-Based KEM for Static-Key Scenarios

Mito¹ is an IND-CCA secure code-based KEM built on the quasi-dyadic algebraic framework introduced above. A key technical contribution of this specification is the formal definition and analysis of the *Quasi-Dyadic Syndrome Decoding* (QDSD) problem—the quasi-dyadic analogue of the well-studied Quasi-Cyclic Syndrome Decoding (QCSD) problem over ring-linear codes. We provide a complete security reduction of Mito.PKE to two concrete variants of the QDSD problem (the $(2, \ell)$ -DQDSD-P and $(3, \ell)$ -DQDSD-PT problems; see [Section 4.1.4](#)), and we prove in [Theorem 4.1.2](#) that the quasi-dyadic structure satisfies the provability conditions required for this reduction to be meaningful.

Similar to HQC, Mito does not require its underlying code family to be computationally indistinguishable from random codes. This property allows the deployment of efficient algebraic structures without undermining the formal security guarantees. Thus, Mito can be equipped with different underlying error-correcting codes while maintaining a unified security level.

Following the standard post-quantum design paradigm, Mito.KEM is constructed by applying a variant of the Fujisaki-Okamoto (FO) transformation [[HHK17](#)] (specifically, the SFO transform [[GHS25](#)]) to an underlying IND-CPA secure encryption scheme, denoted as Mito.PKE.

To optimize both key size and computational efficiency across varying security levels, we define multiple sets of adic parameters, denoted as Mito₁ and Mito₂. The primary advantage of Mito lies in the decryption efficiency of Mito.PKE. By leveraging the inherent properties of the dyadic structure, Mito achieves a substantial reduction in the *equivalent error weight* that must be processed during decoding. Consequently, at the same security level, the *ciphertext size* of Mito is significantly smaller than that of HQC when instantiated with underlying linear codes possessing strong error-correcting capabilities. As a *trade-off*, Mito exhibits a larger public key size and requires increased time for public key generation. Given this profile, Mito is a highly efficient and practical choice for scenarios that use static public keys.

Organization. [Chapter 2](#) provides the necessary mathematical background. [Section 2.2](#) introduces basic coding theory, *adic matrices* and *adic codes* ([Example 2.2.2](#), [Example 2.2.3](#)), and the algebraic structure of dyadic algebras ([Theorem 2.2.1](#)). [Section 2.2.3](#) describes the concatenated Reed-Muller and Reed-Solomon codes used for noise reduction, together with the classical and the *distance-informed erasure decoding* schemes.

[Chapter 3](#) specifies the Mito PKE and KEM schemes. We define all system parameters, auxiliary functions, and sampling algorithms, and provide complete pseudocode for Mito₁ and Mito₂ across KeyGen, Encrypt, Decrypt, Encaps, and Decaps. Detailed parameter sets for all security levels are given in [Section 3.4.1](#).

[Chapter 4](#) contains the security analysis. We begin by formalizing the cryptographic definitions and security assumptions underpinning Mito. We introduce the *fully regular* and *projective regular* scheme frameworks ([Example 4.1.1](#), [Definition 4.1.3](#)), characterize the conditions for a scheme to be provable, and prove in [Theorem 4.1.2](#) that the quasi-dyadic structure satisfies these conditions. We then formalize the (s, ℓ) -DQDSD-P and (s, ℓ) -DQDSD-PT problems ([Definition 4.1.10](#)) and establish the IND-CPA security reduction of Mito.PKE ([Corollary 4.1.1](#)). Building on this foundation, we analyze the decryption failure rate (DFR) of the RSRM concatenated code, evaluate the concrete complexity of information set decoding (ISD) attacks against the underlying QDSD instances, and establish the IND-CCA security of Mito.KEM via the SFO transform.

¹Mito is named after the biological process of “mitosis,” reflecting cell proliferation *without the loss of genetic information*.

2 Mathematical Background

2.1 Notations

We denote by $\mathbb{N} = \{0, 1, 2, \dots\}$ the monoid of natural numbers, and by $[m] = \{0, 1, \dots, m-1\}$ the set of integers modulo m . Let $\mathbb{Z}$ be the ring of integers. For any positive integer q , we denote the quotient ring $\mathbb{Z}/q\mathbb{Z}$ by $\mathbb{Z}_q$. Let $\mathbb{F}_2 = \{0, 1\}$ be the finite field of two elements, and $\mathbb{F}_2^n$ be the n -dimensional vector space over $\mathbb{F}_2$ equipped with linear addition $+$. Let $\mathbb{B}$ be the set of bytes represented as numbers from 0 to 255, i.e., $\mathbb{B} = \{0, 1, 2, \dots, 255\}$. For any set S , let S^m denote the m -fold Cartesian product of S . Specifically, we define

$$S^\infty := \bigcup_{m=1}^{\infty} S^m$$

to represent the union of all finite Cartesian products of S .

Throughout this document, all rings are assumed to be commutative with identity. In particular, we define the base polynomial quotient ring as

$$\mathcal{R}_0 := \mathbb{F}_2[X]/\langle X^n - 1 \rangle.$$

Here, n is an odd prime such that 2 generates the multiplicative group $\mathbb{Z}_n^\times$. This property ensures that the cyclotomic polynomial $\Phi_n(X) = (X^n - 1)/(X - 1)$ is irreducible over $\mathbb{F}_2$. Elements in $\mathcal{R}_0$ are represented as polynomials in the indeterminate X . Standard polynomial addition and multiplication induce corresponding operations over $\mathcal{R}_0$, denoted by $+$ and $*$, respectively. Specifically, for any two elements $a(X) = \sum_{i=0}^{n-1} a_i X^i$ and $b(X) = \sum_{j=0}^{n-1} b_j X^j$ in $\mathcal{R}_0$, their product $c(X) = a(X) * b(X)$ is defined as

$$c(X) = \sum_{k=0}^{n-1} c_k X^k, \quad \text{where} \quad c_k = \sum_{i+j \equiv k \pmod{n}} a_i b_j.$$

We identify elements of $a(X) \in \mathcal{R}_0$ with vectors in $\mathbf{a} \in \mathbb{F}_2^n$ via the canonical mapping

$$\phi : \sum_{i=0}^{n-1} a_i X^i \mapsto (a_0, a_1, \dots, a_{n-1}).$$

Multiplication by X in $\mathcal{R}_0$ induces a linear transformation L_X on $\mathbb{F}_2^n$ that acts as a cyclic shift. For a vector $\mathbf{a} = (a_0, \dots, a_{n-1}) \in \mathbb{F}_2^n$, this is given by

$$L_X(\mathbf{a}) = (a_{n-1}, a_0, \dots, a_{n-2}).$$

For any vector $\mathbf{b} \in \mathbb{F}_2^n$, we define the **circulant matrix** $\text{rot}(\mathbf{b}) \in \mathbb{F}_2^{n \times n}$ where the i -th column corresponds to the i -th cyclic shift of $\mathbf{b}$. Specifically,

$$\text{rot}(\mathbf{b}) := \left(\mathbf{b}^\top \mid L_X(\mathbf{b})^\top \mid \dots \mid L_X^{n-1}(\mathbf{b})^\top \right),$$

where $M^\top$ denotes the transpose of a matrix M . This matrix formulation expresses the polynomial product in $\mathcal{R}_0$ as a vector-matrix multiplication. Identifying polynomials with their row vector representations $\mathbf{a} = \phi(a)$ and $\mathbf{b} = \phi(b)$, we have

$$\mathbf{a} \cdot \text{rot}(\mathbf{b})^\top = \phi(a(X) * b(X)) = \phi(b(X) * a(X)) = \mathbf{b} \cdot \text{rot}(\mathbf{a})^\top,$$

where $\cdot$ denotes standard matrix multiplication over $\mathbb{F}_2$.

Supports and Weights. For an alphabet Σ and $\mathbf{v} \in \Sigma^n$, the **support** of $\mathbf{v}$, denoted by $\text{Supp}_\Sigma(\mathbf{v})$, is the set of indices corresponding to its non-zero coordinates. Accordingly, the Hamming weight of $\mathbf{v} \in \mathbb{F}_2^n$, denoted by $w(\mathbf{v})$, is the cardinality of its support, i.e.,

$$w(\mathbf{v}) = \#\text{Supp}_{\mathbb{F}_2}(\mathbf{v}).$$

We extend these definitions to block vectors. For $\mathbf{v} = (\mathbf{v}_1, \dots, \mathbf{v}_m) \in (\mathbb{F}_2^n)^m$ with $m \in \mathbb{Z}_{\geq 1}$, the weight function returns a **weight vector**

$$\mathbf{w}(\mathbf{v}) := (w(\mathbf{v}_1), w(\mathbf{v}_2), \dots, w(\mathbf{v}_m)) \in \mathbb{N}^m.$$

For a target weight profile $\mathbf{w} \in \mathbb{N}^m$, we define $\mathcal{S}(\mathbf{w})$ as the set of block vectors with weight profile $\mathbf{w}$, i.e.,

$$\mathcal{S}(\mathbf{w}) := \{\mathbf{v} \in (\mathbb{F}_2^n)^m \mid \mathbf{w}(\mathbf{v}) = \mathbf{w}\},$$

and $\mathcal{S}^D(\mathbf{w})$ as the set of block vectors with weight profile $\mathbf{w}$ and **pairwise disjoint supports**, i.e.,

$$\mathcal{S}^D(\mathbf{w}) := \{\mathbf{v} \in (\mathbb{F}_2^n)^m \mid \mathbf{w}(\mathbf{v}) = \mathbf{w} \text{ and } \text{Supp}_{\mathbb{F}_2}(\mathbf{v}_i) \cap \text{Supp}_{\mathbb{F}_2}(\mathbf{v}_j) = \emptyset, \forall i \neq j\}.$$

2.2 Coding Theory

In this section, we introduce the basic coding theory concepts used in our scheme. For a comprehensive survey on coding theory for code-based cryptography, we refer the reader to [WGR22].

Definition 2.2.1 (Linear Codes). A binary $[n, k]$ -linear code $\mathcal{C}$ is a subspace of $\mathbb{F}_2^n$ of dimension k . The code is characterized by a generator matrix $G \in \mathbb{F}_2^{k \times n}$ and a parity-check matrix $H \in \mathbb{F}_2^{(n-k) \times n}$. The structural relationship between $\mathcal{C}$, G , and H is formally described by the following short exact sequence of vector spaces:

$$0 \longrightarrow \mathbb{F}_2^k \xrightarrow{\cdot G} \mathbb{F}_2^n \xrightarrow{\cdot H^\top} \mathbb{F}_2^{n-k} \longrightarrow 0. \quad (2.1)$$

Here, the map $\mathbf{m} \mapsto \mathbf{m}G$ is injective with image $\mathcal{C}$, and the map $\mathbf{c} \mapsto \mathbf{c}H^\top$ is surjective with kernel exactly $\mathcal{C}$. Thus, $\mathcal{C} = \{\mathbf{m}G \mid \mathbf{m} \in \mathbb{F}_2^k\} = \{\mathbf{c} \in \mathbb{F}_2^n \mid \mathbf{c}H^\top = \mathbf{0}\}$.

In particular, for a parity-check matrix H , the vector $\mathbf{s} = \mathbf{x}H^\top$ is called the **syndrome** of $\mathbf{x} \in \mathbb{F}_2^n$ with respect to H .

Metric and error correction. Let $\mathbf{u}, \mathbf{v} \in \mathbb{F}_2^n$. The **Hamming distance** between $\mathbf{u}$ and $\mathbf{v}$ is defined as $d_H(\mathbf{u}, \mathbf{v}) := w(\mathbf{u} + \mathbf{v})$. For a linear code $\mathcal{C}$, the **minimum distance** $d_H = d_H(\mathcal{C})$ is defined as the smallest Hamming distance between any pair of distinct codewords, i.e.,

$$d_H(\mathcal{C}) := \min_{\mathbf{u} \neq \mathbf{v} \in \mathcal{C}} d_H(\mathbf{u}, \mathbf{v}) = \min_{\mathbf{c} \in \mathcal{C} \setminus \{\mathbf{0}\}} w(\mathbf{c}).$$

The maximum error-correcting capability of $\mathcal{C}$, denoted by δ , is determined by the minimum distance d , where $\delta = \lfloor \frac{d-1}{2} \rfloor$. Consequently, a linear code characterized by length n , dimension k , and minimum distance d is formally referred to as an $[n, k, d]$ -code.

Next, we present a family of codes that plays a pivotal role in modern cryptographic schemes.

Definition 2.2.2 (Reproducible Codes). A binary linear code $\mathcal{C}$ is **reproducible** if it can be represented by a generator matrix G or a parity-check matrix H such that G or H is entirely determined by a strict subset of its rows, denoted as the **signature** set $S_{\mathcal{C}}$, and a family of linear transformations $T_{\mathcal{C}}$ acting on $S_{\mathcal{C}}$.

Example 2.2.1 (Quasi-Cyclic Codes). For an integer $s \geq 1$, an $[sn, k]$ -code $\mathcal{C}$ is called **quasi-cyclic** (QC) of index s if, for any codeword $\mathbf{c} = (\mathbf{c}_0, \dots, \mathbf{c}_{s-1}) \in (\mathbb{F}_2^n)^s \cap \mathcal{C}$, the vector obtained by applying a simultaneous circular shift to every block $\mathbf{c}_i$ remains in $\mathcal{C}$. That is, for any $\mathbf{c} \in \mathcal{C}$, it holds that

$$L_X(\mathbf{c}) := (L_X(\mathbf{c}_0), \dots, L_X(\mathbf{c}_{s-1})) \in \mathcal{C}.$$

Consequently, a QC code $\mathcal{C}$ of index s can be viewed as a free $\mathcal{R}_0$ -submodule of $\mathcal{R}_0^s$. Let r denote its rank and

$$\{\mathbf{g}_i(X) = (g_{i,0}(X), \dots, g_{i,s-1}(X)) \mid g_{i,j}(X) \in \mathcal{R}_0, i \in [r], j \in [s]\}$$

be a basis of $\mathcal{C}$. Then, $\mathcal{C}$ is a reproducible code completely determined by the signature set $S_{\mathcal{C}} = \{\mathbf{g}_i(X) \mid i \in [r]\}$ and the family of linear transformations $T_{\mathcal{C}} = \{L_X^k \mid k \in [n]\}$. For convenience, we set $k = rn$. In this case, each circulant block is an $n \times n$ matrix

$$A_{i,j} := \text{rot}(\phi(g_{i,j}(X)))^T \in \mathbb{F}_2^{n \times n}.$$

An **(s, r) -systematic** QC code of index s (with $1 \leq r < s$) is a QC code defined by a parity-check matrix H in systematic form. Let $t = s - r$ denote the number of parity blocks. The code possesses an $tn \times sn$ parity-check matrix H structured as

$$H = (I_{tn} \mid \mathbf{A}) = \begin{pmatrix} I_n & 0 & \dots & 0 & A_{0,0} & \dots & A_{0,r-1} \\ 0 & I_n & \dots & 0 & A_{1,0} & \dots & A_{1,r-1} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & I_n & A_{t-1,0} & \dots & A_{t-1,r-1} \end{pmatrix},$$

where I_{tn} is the identity matrix of size $tn \times tn$, and $\mathbf{A}$ is a block matrix composed of circulant matrices $A_{i,j} \in \mathbb{F}_2^{n \times n}$. $\triangle$

2.2.1 Adic Codes

Let $\mathcal{R}$ be a ring and let $(I, <)$ be a finite totally ordered set. The total order $<$, naturally defined via lexicographical ordering, implies a canonical 2-dimensional array structure indexed by $I \times I$. Consequently, we define an **I -indexed matrix** $\mathcal{H}$ over $\mathcal{R}$ as a collection of entries in $\mathcal{R}$, denoting the entry at position $(\mathbf{i}, \mathbf{j})$ by $H_{\mathbf{i}, \mathbf{j}}$.

Example 2.2.2 (Adic Order and Adic Matrix). Let $q, \ell \geq 1$ be positive integers. The **adic order** on $\mathbb{Z}_q^\ell$ is the total order induced by the bijection $\phi_q : \mathbb{Z}_q^\ell \rightarrow [q^\ell]$, which maps a vector to its integer representation in base q . Specifically, for $\mathbf{a} = (a_0, a_1, \dots, a_{\ell-1}) \in \mathbb{Z}_q^\ell$, this mapping is given by

$$\phi_q(\mathbf{a}) := \sum_{k=0}^{\ell-1} a_k q^k.$$

The ordering on $\mathbb{Z}_q^\ell$ is defined by the condition that $\mathbf{a} < \mathbf{b}$ holds if and only if $\phi_q(\mathbf{a}) < \phi_q(\mathbf{b})$ for any $\mathbf{a}, \mathbf{b} \in \mathbb{Z}_q^\ell$.

Let $\mathcal{H} = (H_{\mathbf{i}, \mathbf{j}})$ be a matrix over a ring $\mathcal{R}$ indexed by the ordered set $\mathbb{Z}_q^\ell$. The matrix $\mathcal{H}$ is called **q -adic** over $\mathcal{R}$ if its entries satisfy the invariance condition $H_{\mathbf{i}, \mathbf{j}} = H_{\mathbf{0}, \mathbf{i} + \mathbf{j}}$ for all indices $\mathbf{i}, \mathbf{j} \in \mathbb{Z}_q^\ell$. Here, the addition $\mathbf{i} + \mathbf{j}$ is performed within the group $\mathbb{Z}_q^\ell$.

Analogous to circulant matrices, a q -adic matrix is uniquely determined by its first row. Let $\mathbf{h} = (h_0, h_1, \dots, h_{q^\ell-1}) \in \mathcal{R}^{q^\ell}$ be a sequence of elements. We denote by $\text{Adic}(\mathbf{h}) = (H_{\mathbf{i}, \mathbf{j}})_{(\mathbf{i}, \mathbf{j}) \in I \times I}$ the unique q -adic matrix defined such that $H_{\mathbf{i}, \mathbf{j}} = h_{\phi_q(\mathbf{i} + \mathbf{j})}$ (see **Figure 2.1**). $\triangle$

$$\begin{array}{ccc}
\mathbf{h}_1 = (h_0, h_1) & \mathbf{h}_2 = (h_0, h_1, h_2) & \mathbf{h}_3 = (h_0, h_1, h_2, h_3) \\
\Downarrow & \Downarrow & \Downarrow \\
\text{Adic}(\mathbf{h}_1) = \begin{pmatrix} h_0 & h_1 \\ h_1 & h_0 \end{pmatrix} & \text{Adic}(\mathbf{h}_2) = \begin{pmatrix} h_0 & h_1 & h_2 \\ h_1 & h_2 & h_0 \\ h_2 & h_0 & h_1 \end{pmatrix} & \text{Adic}(\mathbf{h}_3) = \begin{pmatrix} h_0 & h_1 & h_2 & h_3 \\ h_1 & h_0 & h_3 & h_2 \\ h_2 & h_3 & h_0 & h_1 \\ h_3 & h_2 & h_1 & h_0 \end{pmatrix}
\end{array}$$

Figure 2.1. A 2-adic matrix (left), which degenerates into a standard circulant matrix; a 3-adic matrix (middle), where the row shift corresponds to multiplication by X^2 ; and a 2^2 -adic matrix (right).

It is natural to generalize the exact sequence formulation from (2.1) to the setting of modules over the ring $\mathcal{R}$, giving rise to **ring-linear codes**. However, this proposal does not rely on the advanced algebraic theory of ring-linear codes. Instead, as we will demonstrate in Section 4.1.3, the ring-linear codes defined in our construction are fundamentally linear codes restricted to a finite field alphabet.

Formally, a ring-linear code $\mathcal{C}$ of length N over $\mathcal{R}$ is defined as an $\mathcal{R}$ -submodule of $\mathcal{R}^N$. Assuming $\mathcal{C}$ is a free submodule of rank K , the relationship between the code, its generator matrix $G \in \mathcal{R}^{K \times N}$, and its parity-check matrix $H \in \mathcal{R}^{(N-K) \times N}$ is described by the following exact sequence of $\mathcal{R}$ -modules:

$$0 \longrightarrow \mathcal{R}^K \xrightarrow{\cdot G} \mathcal{R}^N \xrightarrow{\cdot H^\top} \mathcal{R}^{N-K} \longrightarrow 0. \quad (2.2)$$

We continue to refer to G as the generator matrix and H as the parity-check matrix within this ring-theoretic setting.

Example 2.2.3 (Adic and Quasi-Adic Codes over $\mathcal{R}$). For $N = q^\ell$, an $\mathcal{R}$ -linear code $\mathcal{C} \subset \mathcal{R}^N$ (resp. $\mathcal{R}^{sN}$) of length N is called a **q -adic code** (resp. **quasi- q -adic code**) if it admits a generator matrix or a parity-check matrix that is a single q -adic matrix (resp. a block matrix composed entirely of q -adic sub-matrices).

Specifically, for an $[sN, rN]$ -quasi- q -adic code $\mathcal{C}$ with $t = s - r > 0$, its parity-check matrix $\mathcal{H}$ can be partitioned into an $t \times s$ array of sub-matrices $\mathcal{H}^{u,v}$ for $1 \leq u \leq t$ and $1 \leq v \leq s$, with each block $\mathcal{H}^{u,v}$ being an $N \times N$ q -adic matrix over $\mathcal{R}$. Since any q -adic matrix is completely determined by its first row, a quasi- q -adic code is naturally a reproducible code. Its signature set is given by the collection of the first rows of all blocks, i.e.,

$$S_{\mathcal{C}} = \{ \mathbf{h}^u = (\mathbf{h}^{u,0}, \mathbf{h}^{u,1}, \dots, \mathbf{h}^{u,s-1}) \mid \mathcal{H}^{u,v} = \text{Adic}(\mathbf{h}^{u,v}), 1 \leq u \leq t, 1 \leq v \leq s \},$$

and its transformation family $T_{\mathcal{C}}$ consists of the row shifts induced by the index permutation

$$\text{adic}^i : j \mapsto \phi_q \left(\mathbf{i} + \phi_q^{-1}(j) \right).$$

In particular, an **(s, r) -systematic quasi- q -adic code** is defined by a parity-check matrix $\mathcal{H}$ in systematic form

$$\mathcal{H} = \left(I_{tN} \mid \mathcal{A} \right) = \begin{pmatrix} I_N & \dots & 0 & H^{0,0} & \dots & H^{0,r-1} \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & I_N & H^{t-1,0} & \dots & H^{t-1,r-1} \end{pmatrix}. \quad (2.3)$$

△

2.2.2 Dyadic Codes and Dyadic Algebras

It is crucial to observe that for $q \geq 3$, the set of q -adic matrices does not form a commutative ring with identity. A key implication is that the identity matrix I_{q^ℓ} is no longer a q -adic matrix

in such settings, precluding standard systematic constructions. Consequently, this specification restricts its scope exclusively to the case where $q = 2$, which we refer to as the $\mathcal{R}$ -dyadic codes (resp. $\mathcal{R}$ -quasi-dyadic codes).

To describe the algebraic properties of quasi-dyadic codes, we adopt an approach analogous to the polynomial representation of quasi-cyclic codes, establishing an isomorphism between dyadic matrices and a specific polynomial ring. Fixing the base ring $\mathcal{R}_0$, we construct a series of ring extensions by adjoining indeterminates $T_1, T_2, \dots, T_\ell$ subject to the relations $T_i^2 + 1 = 0$ for $i \in \{1, \dots, \ell\}$. We denote the resulting multivariate quotient ring as

$$\mathcal{R}_\ell := \mathcal{R}_0[T_1, T_2, \dots, T_\ell] / \langle T_1^2 + 1, T_2^2 + 1, \dots, T_\ell^2 + 1 \rangle.$$

Similar to $\mathcal{R}_0$, the addition and multiplication of $\mathcal{R}_\ell$ are induced by the natural operations on $\mathbb{F}_2[X, T_1, \dots, T_\ell]$, denoted as $+$ and $*$. The injection $\iota : \mathcal{R}_i \rightarrow \mathcal{R}_{i+1}$ defined by $r \mapsto r + 0 \cdot T_{i+1}$ induces a tower of rings $\mathcal{R}_0 \subset \mathcal{R}_1 \subset \mathcal{R}_2 \subset \dots \subset \mathcal{R}_\ell$. For $r \in \mathcal{R}_{i+1}$, there exists a unique pair $(r_0, r_1) \in \mathcal{R}_i^2$ such that $r = r_0 + r_1 * T_{i+1}$. Therefore, for $y = (y_0, y_1), h = (h_0, h_1) \in \mathcal{R}_i^2$, the equation $y * h = z$ on $\mathcal{R}_{i+1}$ can be represented as a system of equations

$$y_0 * h_0 + y_1 * h_1 = z_0, \quad y_0 * h_1 + y_1 * h_0 = z_1$$

on $\mathcal{R}_i$. This establishes a bijection between solutions in the extended ring and the underlying system (e.g., see Figure 2.2), which we refer to as the *dyadic decomposition*.

$$\begin{array}{c}
y, h \in \mathcal{R}_2 \implies \begin{array}{l} y = y_0 + y_1 * T_2 \\ h = h_0 + h_1 * T_2 \end{array} \implies \begin{array}{l} y_0 = y_{00} + y_{01} * T_1, \quad y_1 = y_{10} + y_{11} * T_1 \\ h_0 = h_{00} + h_{01} * T_1, \quad h_1 = h_{10} + h_{11} * T_1 \end{array} \\
\implies \begin{array}{l} y = y_{00} + y_{01} * T_1 + y_{10} * T_2 + y_{11} * T_1 T_2 \\ h = h_{00} + h_{01} * T_1 + h_{10} * T_2 + h_{11} * T_1 T_2 \end{array} \\
\\
y * h = z \implies \begin{pmatrix} h_0 & h_1 \\ h_1 & h_0 \end{pmatrix} * \begin{pmatrix} y_0 \\ y_1 \end{pmatrix} = \begin{pmatrix} z_0 \\ z_1 \end{pmatrix} \implies \begin{pmatrix} h_{00} & h_{01} & h_{10} & h_{11} \\ h_{01} & h_{00} & h_{11} & h_{10} \\ h_{10} & h_{11} & h_{00} & h_{01} \\ h_{11} & h_{10} & h_{01} & h_{00} \end{pmatrix} * \begin{pmatrix} y_{00} \\ y_{01} \\ y_{10} \\ y_{11} \end{pmatrix} = \begin{pmatrix} z_{00} \\ z_{01} \\ z_{10} \\ z_{11} \end{pmatrix} \\
\implies \text{Adic}(\mathbf{h}) * \mathbf{y}^\top = \mathbf{z}^\top
\end{array}$$

Figure 2.2. Decomposition of dyadic multiplication across different levels from $\mathcal{R}_2$ to $\mathcal{R}_0$.

Forgetting the multiplicative structure, we identify $\mathcal{R}_\ell$ with the $\mathbb{F}_2^n$ -vector space $(\mathbb{F}_2^n)^{2^\ell}$ via the coefficient isomorphism. Specifically, the set of monomials

$$\left\{ \mathbf{T}^\alpha = \prod_{j=1}^{\ell} T_j^{\alpha_j} \mid (\alpha_1, \dots, \alpha_\ell) \in \mathbb{F}_2^\ell \right\}$$

forms a basis for $\mathcal{R}_\ell$ viewed as a module over $\mathcal{R}_0$.

Let $\mathcal{D}$ denote the set of $2^\ell \times 2^\ell$ dyadic matrices over $\mathcal{R}_0$. Since any matrix $H \in \mathcal{D}$ is uniquely determined by its first row $\mathbf{h} = (h_0, h_1, \dots, h_{2^\ell-1})$, we define a mapping $\Psi : \mathcal{D} \rightarrow \mathcal{R}_\ell$ that maps a dyadic matrix to a polynomial whose coefficients correspond to the first row of the matrix. Specifically, for a matrix H with first row $\mathbf{h}$,

$$\Psi(H) = \sum_{\alpha \in \mathbb{F}_2^\ell} h_{\phi_2(\alpha)} \mathbf{T}^\alpha.$$

It is straightforward to verify that the mapping Ψ constitutes an isomorphism of $\mathcal{R}_0$ -algebras. This structural equivalence allows us to translate operations from the matrix domain directly into the polynomial domain. Consequently, we state the following theorem.

Theorem 2.2.1. The $\mathcal{R}_0$ -algebra of $2^\ell \times 2^\ell$ dyadic matrices over the base ring $\mathcal{R}_0$ is isomorphic to the dyadic algebra $\mathcal{R}_\ell$.

2.2.3 Concatenated Reed-Muller and Reed-Solomon Codes

For an $\mathcal{R}$ -linear code $\mathcal{C}$, if the underlying ring $\mathcal{R}$ is isomorphic to a finite field $\mathbb{F}_{2^k}$, we treat it as a linear code over the extended binary alphabet $\mathbb{F}_{2^k}$. Its parameters are denoted by $[N, K, D]_{2^k}$, corresponding to the code length N , dimension K over $\mathbb{F}_{2^k}$, and minimum Hamming distance D induced by $w(\mathbf{v}) = \text{supp}_{\mathbb{F}_{2^k}}(\mathbf{v})$, respectively.

To guarantee highly efficient error correction alongside cryptographic hardness, we employ a concatenated code structure. Let $\mathcal{C}_{\text{out}}$ be an outer linear code with parameters $[N_{\text{out}}, K_{\text{out}}, D_{\text{out}}]_{2^k}$, and $\mathcal{C}_{\text{in}}$ be an inner binary linear code with parameters $[n_{\text{in}}, k_{\text{in}}, d_{\text{in}}]$, where the dimensions $k_{\text{in}} = k$. Denote G_{out} (resp. G_{in}) and H_{out} (resp. H_{in}) as the generator and parity-check matrices of $\mathcal{C}_{\text{out}}$ (resp. $\mathcal{C}_{\text{in}}$). The **concatenated code** $\mathcal{C} := \mathcal{C}_{\text{out}} \circ \mathcal{C}_{\text{in}}$ is defined by the exact sequence

$$\begin{array}{ccccccc}
 0 & \longrightarrow & \mathbb{F}_{2^k}^{K_{\text{out}}} & \xrightarrow{\cdot G_{\text{out}}(I_{N_{\text{out}}} \otimes_k G_{\text{in}})} & \mathbb{F}_2^{n_{\text{in}} N_{\text{out}}} & \xrightarrow{\cdot H_{\text{cot}}^\top} & \mathbb{F}_2^{n_{\text{in}} N_{\text{out}} - k_{\text{in}} K_{\text{out}}} \longrightarrow 0 \\
 & & \downarrow \cdot G_{\text{out}} & & \uparrow \cdot (I_{N_{\text{out}}} \otimes_k G_{\text{in}}) & & \\
 & & \mathbb{F}_{2^k}^{N_{\text{out}}} & \xrightarrow{\sim} & (\mathbb{F}_2^k)^{N_{\text{out}}} & &
 \end{array}$$

Here, the middle equivalence denotes the standard basis embedding from the extension field to the binary vector space, $\otimes_k$ is the **Kronecker product of matrices**, and

$$H_{\text{cot}} = \begin{pmatrix} I_{N_{\text{out}}} \otimes_k H_{\text{in}} \\ \varpi(H_{\text{out}})(I_{N_{\text{out}}} \otimes_k E_{\text{in}}^\top) \end{pmatrix}$$

where E_{in} is the inner recovery matrix satisfying $G_{\text{in}} E_{\text{in}} = I_k$, and $\varpi(H_{\text{out}})$ is the binary expansion of the outer parity-check matrix.

Asymptotically, random concatenated codes can meet the **Gilbert-Varshamov** (GV) bound [BZ73], even when the outer code is a fixed **Maximum Distance Separable** (MDS) code [Tho03]. Following [AMAD⁺24], our proposal instantiates the inner code as a duplicated **Reed-Muller** (RM) code and employs a shortened **Reed-Solomon** (RS) code as the outer code to match the required payload and security level.

Duplicated Reed-Muller Codes

RM codes are among the oldest families of error-correcting codes, having found widespread application across various communication systems and computer science domains. For a modern introduction to the underlying theory of Reed-Muller codes, we refer the reader to [ASY20].

Formally, for $f \in \mathbb{F}_2[x_0, x_1, \dots, x_{m-1}]$ and $z = (z_0, z_1, \dots, z_{m-1}) \in \mathbb{F}_2^m$, denote

$$\text{Eval}_z(f) := f(z_0, z_1, \dots, z_{m-1}) \in \mathbb{F}_2$$

as the evaluation of f at the point z . We define

$$\text{Eval}(f) := (\text{Eval}_z(f) : z \in \mathbb{F}_2^m) \in (\mathbb{F}_2)^{2^m}$$

as the vector of evaluations of f across all 2^m vectors $z \in \mathbb{F}_2^m$. The *r -th order binary Reed-Muller code*, denoted as $\text{RM}(r, m)$, consists of the evaluation vectors of all polynomials in

$$\mathbb{F}_2[x_0, x_1, \dots, x_{m-1}]_{\leq r} := \{f \in \mathbb{F}_2[x_0, x_1, \dots, x_{m-1}] \mid \deg(f) \leq r\}.$$

It is well-known that $\text{RM}(r, m)$ forms a binary linear code with parameters $[2^m, k, 2^{m-r}]$, where $k = \sum_{i=0}^r \binom{m}{i}$. Here, $\mathbb{F}_2^k$ is equipped with a standard basis

$$\left\{ \prod_{i \in I} x_i \mid I \subseteq [m], \#I \leq r \right\}.$$

Note that any such polynomial can be uniquely decomposed as

$$f(x_1, \dots, x_m) = g(x_1, \dots, x_{m-1}) + x_m \cdot h(x_1, \dots, x_{m-1}),$$

where $\deg(g) \leq r$ and $\deg(h) \leq r - 1$. This algebraic structure gives rise to a split short exact sequence

$$0 \longrightarrow \text{RM}(r, m-1) \xrightarrow{\iota} \text{RM}(r, m) \xrightarrow[\pi \circ \sigma]{\pi} \text{RM}(r-1, m-1) \longrightarrow 0, \quad (2.4)$$

where the inclusion map is defined as $\iota : \mathbf{u} \mapsto (\mathbf{u}, \mathbf{u})$, the projection map as $\pi : (\mathbf{a}, \mathbf{b}) \mapsto \mathbf{a} + \mathbf{b}$, and the section map as $\sigma : \mathbf{v} \mapsto (\mathbf{0}, \mathbf{v})$. The sequence (2.4) implies a direct sum decomposition,

$$\text{RM}(r, m) \cong \text{Im}(\iota) \oplus \text{Im}(\sigma),$$

which translates naturally into the recursive block structure of the generator matrix. Let $G_{\text{RM}(r, m)}$ denote the generator matrix of $\text{RM}(r, m)$. The basis vectors from $\text{Im}(\iota)$ form the top blocks $(G_{\text{RM}(r, m-1)} \mid G_{\text{RM}(r, m-1)})$, while the basis vectors from $\text{Im}(\sigma)$ form the bottom blocks $(\mathbf{0} \mid G_{\text{RM}(r-1, m-1)})$. Stacking these subspaces yields the standard $(\mathbf{u}, \mathbf{u} + \mathbf{v})$ recursive construction

$$G_{\text{RM}(r, m)} = \begin{pmatrix} G_{\text{RM}(r, m-1)} & G_{\text{RM}(r, m-1)} \\ \mathbf{0}_{2^{m-1}} & G_{\text{RM}(r-1, m-1)} \end{pmatrix}.$$

Consequently, for any r and m , the generator matrix of $\text{RM}(r, m)$ can be derived recursively. On the other hand, it is straightforward to deduce from (2.4) that the dual code of an RM code remains an RM code. That is,

$$\text{RM}(r, m)^\perp = \text{RM}(m-1-r, m).$$

Therefore, the aforementioned recursive construction is equally applicable to the parity-check matrix $H_{\text{RM}(r, m)} = G_{\text{RM}(m-1-r, m)}$. Specifically, for the first-order Reed-Muller code $\text{RM}(1, 7)$ utilized in our context, we substitute $r = 1$ and $m = 7$. Noting that $G_{\text{RM}}(0, m) = (\mathbf{1}^{2^m})$, the generator matrix can be precisely expressed as

$$G_{\text{RM}(1, m)} = \begin{pmatrix} G_{\text{RM}(1, m-1)} & G_{\text{RM}(1, m-1)} \\ \mathbf{0}_{2^{m-1}} & \mathbf{1}^{2^{m-1}} \end{pmatrix},$$

which provides a highly efficient recursive framework originating from

$$G_{\text{RM}(1, 1)} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}.$$

Duplicated RM codes and FHT. A *repetition code* $\text{Rep}(t)$ of length t encodes a single bit by repeating it t times. Its generator matrix is $G_{\text{Rep}(t)} = \mathbf{1}^t$, and its parity-check matrix H_{rep} of dimension $(t-1) \times t$ checks the equality of adjacent bits.

The *t -duplicated RM code*, denoted as $t\text{-RM}(1, 7)$, is formed by repeating each bit of a codeword in $\text{RM}(1, 7)$ exactly t times. Hence, the total length of this new code becomes $n = 128t$, while its information dimension remains $k = 8$. Formally, the generator matrix G of $t\text{-RM}(1, 7)$ can be precisely constructed via

$$G_{t\text{-RM}(1,7)} = G_{\text{RM}(1,7)} \otimes_k G_{\text{Rep}(t)}.$$

To validate this duplicated codeword, the parity-check matrix H of $t\text{-RM}(1, 7)$ must simultaneously enforce two conditions. First, it must guarantee the internal consistency of the bits within each of the 128 length- t blocks. Second, the values extracted from these 128 blocks must satisfy the standard $\text{RM}(1, 7)$ parity-check rules. We explicitly construct H as a block matrix

$$H = \begin{pmatrix} I_{128} \otimes_k H_{\text{Rep}(t)} \\ H_{\text{RM}(1,7)} \otimes_k \mathbf{e}_1 \end{pmatrix},$$

where I_{128} is the 128×128 identity matrix, and $\mathbf{e}_1 = (1, 0, \dots, 0)$ is a length- t row vector. It is easy to verify the following sequence is exact:

$$0 \longrightarrow \mathbb{F}_2^8 \xrightarrow{\cdot G} \mathbb{F}_2^{128t} \xrightarrow{\cdot H^\top} \mathbb{F}_2^{128t-8} \longrightarrow 0.$$

To decode $t\text{-RM}(1, 7)$, we first aggregate the received signals for each repeated block following [GAMA⁺25], and then apply the soft-decision **Fast Hadamard Transform** (FHT) decoder.

In general, our decoding algorithm relies on the **Maximum Likelihood** (ML) principle. Consider a binary-input memoryless channel $W : \{0, 1\} \rightarrow \mathcal{Y}$. The **log-likelihood ratio** (LLR) of an output symbol $y \in \mathcal{Y}$ is defined as

$$\text{LLR}(y) := \ln \left(\frac{W(y|0)}{W(y|1)} \right),$$

where the transition probability $W(y|\epsilon)$ is the conditional probability of obtaining the channel output $y \in \mathcal{Y}$ when the channel input is $\epsilon \in \{0, 1\}$.

Let $y = (y_{z,j} : z \in \mathbb{F}_2^7, j \in [t])$ denote the received sequence after transmitting a $t\text{-RM}(1, 7)$ codeword c over the memoryless channel. Here, for a given coordinate $z \in \mathbb{F}_2^7$, $y_{z,j}$ represents the j -th received copy associated with the same coordinate z of the underlying $\text{RM}(1, 7)$ codeword. We compute the *aggregated LLR* for each coordinate z by summing the LLRs over the t repetitions, i.e.,

$$L_z := \sum_{j=1}^t \text{LLR}(y_{z,j}).$$

Once these 2^7 aggregated LLRs are computed, the decoding problem reduces to the ML decoding of the first-order RM code $\text{RM}(1, 7)$ [Gre66, BS86].

Given the aggregated LLR vector $(L_z : z \in \mathbb{F}_2^7)$, the ML decoder aims to find $c = (c_z : z \in \mathbb{F}_2^7) \in \text{RM}(1, 7)$ to maximize the likelihood $\prod_{z \in \mathbb{F}_2^7} W(y_z|c_z)$ for a given $y = (y_z : z \in \mathbb{F}_2^7)$, which is equivalent to maximizing the quantity

$$\sum_{z \in \mathbb{F}_2^7} \ln \left(\frac{W(y_z|c_z)}{\sqrt{W(y_z|0)W(y_z|1)}} \right).$$

Since the codeword c is a binary vector, the term inside the sum simplifies to

$$\ln \left(\frac{W(y_z|c_z)}{\sqrt{W(y_z|0)W(y_z|1)}} \right) = \begin{cases} \frac{1}{2} L_z & \text{if } c_z = 0, \\ -\frac{1}{2} L_z & \text{if } c_z = 1. \end{cases}$$

Thus, the ML decoding task is equivalent to finding $c \in \text{RM}(1, 7)$ that maximizes

$$\frac{1}{2} \sum_{z \in \mathbb{F}_2^7} (-1)^{c_z} L_z.$$

By definition, every $c \in \text{RM}(1, 7)$ corresponds to a polynomial $u_{-1} + \sum_{i=0}^6 u_i x_i$ of degree one. Hence, we have $c_z = u_{-1} + \sum_{i=0}^6 u_i z_i$, where $z_0, z_1, \dots, z_6$ are the coordinates of the vector z . Our task is to find $u_{-1}, u_0, u_1, \dots, u_6 \in \mathbb{F}_2$ to maximize

$$\sum_{z \in \mathbb{F}_2^7} (-1)^{(u_{-1} + \sum_{i=0}^6 u_i z_i)} L_z = (-1)^{u_{-1}} \sum_{z \in \mathbb{F}_2^7} (-1)^{\sum_{i=0}^6 u_i z_i} L_z.$$

For a binary vector $u = (u_0, u_1, \dots, u_6) \in \mathbb{F}_2^7$, we define

$$\widehat{L}(u) := \sum_{z \in \mathbb{F}_2^7} (-1)^{\sum_{i=0}^6 u_i z_i} L_z.$$

To find the maximizer, we only need to compute $\widehat{L}(u)$ for all $u \in \mathbb{F}_2^7$. Crucially, the vector $(\widehat{L}(u) : u \in \mathbb{F}_2^7)$ is exactly the Hadamard Transform of the vector $(L_z : z \in \mathbb{F}_2^7)$.

Once the values of $(\widehat{L}(u) : u \in \mathbb{F}_2^7)$ are computed, we find the index vector $u^* = (u_0^*, \dots, u_6^*) \in \mathbb{F}_2^7$ that maximizes $|\widehat{L}(u)|$. If $\widehat{L}(u^*) > 0$, the decoder outputs the codeword corresponding to $u_{-1} = 0$ along with $(u_0^*, \dots, u_6^*)$. Otherwise, it sets $u_{-1} = 1$. This completes the soft-decision FHT decoder (see the general case in [Algorithm 1](#)), which leads to the following general theorem:

Theorem 2.2.2. The FHT decoder finds the ML decoding result in $O(n \log n)$ time when decoding first-order RM codes $\text{RM}(1, m)$ for $n = 2^m$.

Algorithm 1 FHT Decoder for t -RM(1, m)

Input: The LLR matrix $(\text{LLR}(y_{z,j}) : z \in \mathbb{F}_2^m, j \in [t])$.

Output: A message $m \in \mathbb{F}_2^{m+1}$.

```

1:  $L_z \leftarrow \sum_{j=1}^t \text{LLR}(y_{z,j})$  for all  $z \in \mathbb{F}_2^m$  /* Aggregate LLRs over  $t$  duplications */
2:  $(\widehat{L}(u) : u \in \mathbb{F}_2^m) \leftarrow \text{FHT}(L_z : z \in \mathbb{F}_2^m)$  /* Compute the Fast Hadamard Transform */
3:  $u^* = (u_0^*, \dots, u_{m-1}^*) \leftarrow \arg \max_{u \in \mathbb{F}_2^m} |\widehat{L}(u)|$  /* Find the index that maximizes the absolute correlation */
4:  $\text{if } \widehat{L}(u^*) > 0 \text{ then}$  /* Recover the codeword based on the sign of the peak */
5:    $m \leftarrow (0, u_0^*, \dots, u_{m-1}^*)$ 
6:  $\text{else}$ 
7:    $m \leftarrow (1, u_0^*, \dots, u_{m-1}^*)$ 
8: return  $m$ 
```

Shortened Reed-Solomon Codes

RS codes arise as a special case of algebraic geometry codes defined over rational curves of genus 0. While RM codes are generated by evaluating multivariate, low-degree polynomials over all points in $\mathbb{F}_2^m$, RS codes are analogously generated by evaluating univariate, high-degree polynomials over the non-zero points of $\mathbb{F}_q$, where $q = 2^m$.

Specifically, an RS code with parameters $[N, K, N - K + 1]_q$ is defined over the finite field $\mathbb{F}_q$. We select a set of N distinct non-zero elements $S = \{\alpha_1, \alpha_2, \dots, \alpha_N\}$ from the field $\mathbb{F}_q$ as the evaluation

point set. The code space $\text{RS}(N, K)_q$ consists of the evaluations of all univariate polynomials $f(x) \in \mathbb{F}_q[x]$ of degree strictly less than K , i.e.,

$$\text{RS}(N, K)_q := \{(f(\alpha_1), f(\alpha_2), \dots, f(\alpha_N)) \mid f \in \mathbb{F}_q[x], \deg(f) < K\}.$$

Since the number of roots of a univariate polynomial cannot exceed its degree, any two polynomials of degree less than K can agree on at most $K - 1$ points. Consequently, the minimum distance of the RS code is $D = N - (K - 1) = N - K + 1$, directly attaining the Singleton bound and thus establishing RS codes as MDS codes.

In the following, we choose the evaluation set S to be consecutive powers of a primitive element α of $\mathbb{F}_q^\times$, namely $S = \{1, \alpha, \alpha^2, \dots, \alpha^{N-1}\}$ where $N = q - 1$. The resulting RS code acquires a strict cyclic structure. To see this, consider a codeword $c = (f(1), f(\alpha), \dots, f(\alpha^{N-1}))$ generated by a polynomial $f(x)$ with $\deg(f) < K$. A cyclic right-shift of c yields $c' = (f(\alpha^{N-1}), f(1), \dots, f(\alpha^{N-2}))$. Since $\alpha^N = 1$, we can define a scaled polynomial $\tilde{f}(x) = f(\alpha^{-1}x)$. Evaluating $\tilde{f}(x)$ sequentially over the set S exactly produces the shifted sequence c' . Because scaling the variable does not increase the polynomial degree, we have $\deg(\tilde{f}) = \deg(f) < K$, meaning that c' is a valid codeword in $\text{RS}(N, K)_q$.

By the fundamental theorem of cyclic codes, any such linear cyclic code of length N corresponds to an ideal in the quotient ring $\mathbb{F}_q[x]/\langle x^N - 1 \rangle$. Consequently, the code space is determined by a unique monic generator polynomial $g(x)$ that divides $x^N - 1$. Assuming the roots of $g(x)$ are $\alpha, \alpha^2, \dots, \alpha^{2\sigma}$ for $D = 2\sigma + 1$, then its degree is exactly $N - K$, i.e.,

$$g(x) = \prod_{i=1}^{2\sigma} (x - \alpha^i) = g_0 + g_1x + g_2x^2 + \dots + g_{N-K}x^{N-K}.$$

Since any codeword polynomial $c(x)$ must be a multiple of $g(x)$, we have $c(x) = m(x)g(x)$ for some message polynomial $m(x)$ with $\deg(m) < K$. Therefore, multiplying $g(x)$ successively by $1, x, x^2, \dots, x^{K-1}$ yields a basis for the code space. The corresponding non-systematic generator matrix G is structured by stacking the shifted vectors as rows, i.e.,

$$G = \begin{pmatrix} \mathbf{g} \\ L_X(\mathbf{g}) \\ \vdots \\ L_X^{K-1}(\mathbf{g}) \end{pmatrix}$$

where $\mathbf{g} = (g_0, g_1, \dots, g_{N-K}, 0, \dots, 0) \in \mathbb{F}_q^N$.

On the other hand, since $c(x)$ is a multiple of $g(x)$, all roots of $g(x)$ must be roots of $c(x)$. Thus, for any codeword $c = (c_0, c_1, \dots, c_{N-1})$, its corresponding polynomial must satisfy $c(\alpha^i) = 0$ for $i = 1, 2, \dots, N - K$. This algebraic constraint directly gives rise to the parity-check matrix H with its characteristic Vandermonde structure

$$H = \begin{pmatrix} 1 & \alpha & \alpha^2 & \dots & \alpha^{N-1} \\ 1 & \alpha^2 & (\alpha^2)^2 & \dots & (\alpha^2)^{N-1} \\ 1 & \alpha^3 & (\alpha^3)^2 & \dots & (\alpha^3)^{N-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha^{N-K} & (\alpha^{N-K})^2 & \dots & (\alpha^{N-K})^{N-1} \end{pmatrix}.$$

In conclusion, the cyclic RS code $\text{RS}(N, K)_q$ is determined by the following commutative exact sequence diagram, which bridges its linear vector space representation with its univariate

polynomial ring isomorphism:

$$\begin{array}{ccccccc}
0 & \longrightarrow & \mathbb{F}_q^K & \xrightarrow{\cdot G} & \mathbb{F}_q^N & \xrightarrow{\cdot H^\top} & \mathbb{F}_q^{N-K} \longrightarrow 0 \\
& & \downarrow \cong & & \downarrow \cong & & \downarrow = \\
0 & \longrightarrow & \mathbb{F}_q[x]_{<K} & \xrightarrow{\cdot g(x)} & \mathbb{F}_q[x]/\langle x^N - 1 \rangle & \xrightarrow{\text{Eval}_S} & \mathbb{F}_q^{2\sigma} \longrightarrow 0
\end{array}$$

Here, Eval_S denotes the multi-point evaluation map at the root set of the generator polynomial, i.e.,

$$\text{Eval}_S(f) := (f(\alpha^i) : i = 1, \dots, 2\sigma).$$

Shortening RS Codes. In particular, the length of the message may not align with the standard dimension K of a given RS code. To address this, we employ the shortening operation.

Let $S = \{\alpha_1, \dots, \alpha_N\}$ be the evaluation point set for the original code $\text{RS}(N, K)_q$. To shorten this code by s symbols, we select a subset of evaluation points $P \subset S$ with $\#P = s$. The shortening operation restricts the original code to the subspace of codewords that evaluate to exactly 0 at all coordinates in P , and subsequently truncates these deterministic zeros.

Algebraically, the condition $f(\beta) = 0$ for all $\beta \in P$ forces the message polynomial to be divisible by the vanishing polynomial of P , defined as

$$Z_P(x) = \prod_{\beta \in P} (x - \beta).$$

Thus, any such polynomial factors as $f(x) = Z_P(x) \cdot h(x)$. Given $\deg(f) < K$, it strictly follows that $\deg(h) < K - s$. The **shortened RS code**, denoted as $s\text{-SRS}(N, K)_q$, is generated by evaluating $h(x)$ over the remaining $N - s$ points in $S \setminus P$. Its parameters are naturally $[N - s, K - s, D']_q$. Since deleting coordinates with value 0 does not decrease the Hamming weight of any non-zero codeword, the minimum distance satisfies $D' \geq N - K + 1$. However, the Singleton bound restricts

$$D' \leq (N - s) - (K - s) + 1 = N - K + 1.$$

Consequently, $D' = N - K + 1$, proving that the shortened code preserves the MDS property. The shortened code forms the kernel of the evaluation map Eval_P at the subset P . This algebraic structure is captured by the following commutative diagram, linking the univariate polynomial rings to the finite field vector spaces:

$$\begin{array}{ccccccc}
0 & \longrightarrow & \mathbb{F}_q[x]_{<K-s} & \xrightarrow{\cdot Z_P(x)} & \mathbb{F}_q[x]_{<K} & \xrightarrow{\text{Eval}_P} & \mathbb{F}_q^s \longrightarrow 0 \\
& & \downarrow \text{Eval}_{S \setminus P} & & \downarrow \text{Eval}_S & & \downarrow = \\
0 & \longrightarrow & s\text{-SRS}(N, K)_q & \xrightarrow{\iota} & \text{RS}(N, K)_q & \xrightarrow{\pi_P} & \mathbb{F}_q^s \longrightarrow 0
\end{array}$$

Here, π_P denotes the projection onto the coordinates in P . The embedding map ι injects the shortened codeword generated by $h(x)$ back into the ambient space by padding zeros, which equates to the evaluation of $Z_P(x)h(x)$.

Encoding Shortened RS Codes. From a matrix perspective, if the parent code is defined by a systematic generator matrix $G = (I_K \mid A)$, encoding $s\text{-SRS}(N, K)_q$ is straightforward: one pads the $(K - s)$ -symbol message with s zeros, multiplies it by G , and truncates the s deterministic zero positions from the resulting codeword.

We follow the efficient implementation approach for shortened RS codes as described in [GAMA⁺25, Section 3.4.2]. Specifically, we view the codeword space $\text{RS}(N, K)_q$ as the ideal in $\mathbb{F}_q[x]/\langle x^N - 1 \rangle$ consisting of all polynomials that are divisible by the generator polynomial $g(x)$.

For a message vector

$$\mathbf{u} = (u_0, u_1, \dots, u_{K-s-1}) \in \mathbb{F}_q^{K-s}$$

of length $K - s$, we map it to its corresponding polynomial $u(x) = \sum_{i=0}^{K-s-1} u_i x^i$. To achieve a systematic representation without constructing an explicit generator matrix, the encoding proceeds in the following **Algorithm 2**:

Algorithm 2 Systematic Encoding of s -SRS(N, K) $_q$

Input: Shortening parameter s , generator polynomial $g(x)$, and message vector $\mathbf{u}$.

Output: Codeword vector $\mathbf{c} \in \mathbb{F}_q^{N-s}$.

- 1: $(u_0, u_1, \dots, u_{K-s-1}) \leftarrow \mathbf{u}$ */* Map the message vector to a polynomial representation */*
 - 2: $u(x) \leftarrow \sum_{i=0}^{K-s-1} u_i x^i$ */* Shift the message to higher-degree positions */*
 - 3: $u_{\text{shift}}(x) \leftarrow x^{N-K} \cdot u(x)$ */* Compute the parity-check polynomial via polynomial division */*
 - 4: $b(x) \leftarrow u_{\text{shift}}(x) \pmod{g(x)}$ */* Combine the remainder and the shifted message */*
 - 5: $c(x) \leftarrow b(x) + u_{\text{shift}}(x)$ */* Extract the valid $N - s$ coefficients (implicit truncation) */*
 - 6: Parse $c(x)$ as $\sum_{i=0}^{N-s-1} c_i x^i$
 - 7: $\mathbf{c} \leftarrow (c_0, c_1, \dots, c_{N-s-1})$
 - 8: **return** $\mathbf{c}$
-

By construction, $c(x)$ is a multiple of $g(x)$, ensuring it is a valid codeword of length $N - s$ in the code space of s -SRS(N, K) $_q$. Once a valid codeword $\mathbf{c}$ is recovered, the underlying message $\mathbf{u}$ can be extracted from the rightmost $K - s$ positions of the codeword.

Decoding RS and Shorten RS Codes. Since the receiver can treat the shortened RS code as a standard RS codeword via zero-padding, the shortened RS code and the original RS code utilize the same decoding algorithm. Let $c(x)$ be the transmitted codeword polynomial and $r(x) = c(x) + e(x)$ be the received polynomial. Assume that ν errors occurred during channel transmission, where $\nu \leq \sigma$. Let the error positions be $i_1, i_2, \dots, i_\nu$, and their corresponding error values be $Y_1, Y_2, \dots, Y_\nu$. We define the **error locators** as $X_l = \alpha^{i_l}$. The algebraic decoding proceeds in three distinct steps:

Step 1: Syndrome Computation

Evaluate the received polynomial $r(x)$ at the roots $\alpha, \dots, \alpha^{2\sigma}$ to obtain 2σ syndromes

$$S_j = r(\alpha^j) = c(\alpha^j) + e(\alpha^j) = e(\alpha^j) = \sum_{l=1}^{\nu} Y_l X_l^j \quad \text{for } j = 1, 2, \dots, 2\sigma.$$

Using these values, we construct the **syndrome polynomial**

$$S(x) = \sum_{j=1}^{2\sigma} S_j x^{j-1}.$$

Step 2: Finding the Error Locations

Let

$$\Lambda(x) = \prod_{l=1}^v (1 - X_l x)$$

be the *error locator polynomial*. By expanding this polynomial with respect to the variable x as

$$\Lambda(x) = 1 + \sum_{l=1}^v \Lambda_l x^l,$$

we can determine the positions of the errors in $r(x)$ by finding its roots. To compute its coefficients, we employ the **Berlekamp-Massey** (BM) algorithm, which iteratively determines the valid $\Lambda(x)$ of minimal degree. The algorithm initializes with $\Lambda^{(0)}(x) = 1$, an auxiliary polynomial $B^{(0)}(x) = 1$, and a length scalar $L_0 = 0$, and updates the state as detailed in [Algorithm 3](#).

Algorithm 3 Berlekamp-Massey Algorithm

Input: Syndromes $S_1, S_2, \dots, S_{2\sigma}$.

Output: The locator polynomial $\Lambda(x)$.

```

1:  $\Lambda^{(0)}(x) \leftarrow 1$ 
2:  $B^{(0)}(x) \leftarrow 1$ 
3:  $L_0 \leftarrow 0$ 
4: for  $r = 1, \dots, 2\sigma$  do
5:    $\Delta_r \leftarrow S_r + \sum_{j=1}^{L_{r-1}} \Lambda_j^{(r-1)} S_{r-j}$ 
6:   if  $\Delta_r = 0$  then
7:      $\Lambda^{(r)}(x) \leftarrow \Lambda^{(r-1)}(x)$ 
8:      $B^{(r)}(x) \leftarrow x \cdot B^{(r-1)}(x)$ 
9:      $L_r \leftarrow L_{r-1}$ 
10:  else
11:     $\Lambda^{(r)}(x) \leftarrow \Lambda^{(r-1)}(x) - \Delta_r \cdot x \cdot B^{(r-1)}(x)$ 
12:    if  $2L_{r-1} \leq r - 1$  then
13:       $B^{(r)}(x) \leftarrow \Delta_r^{-1} \cdot \Lambda^{(r-1)}(x)$ 
14:       $L_r \leftarrow r - L_{r-1}$ 
15:    else
16:       $B^{(r)}(x) \leftarrow x \cdot B^{(r-1)}(x)$ 
17:       $L_r \leftarrow L_{r-1}$ 
18: return  $\Lambda^{(2\sigma)}(x)$ 
```

Step 3: Error Value Computation

Once the error locations $X_l = \alpha^{i_l}$ are identified, we determine their specific magnitudes Y_l . First, we compute the *full error evaluator polynomial*

$$\Omega(x) \equiv (\Lambda(x)S(x)) \pmod{x^{2\sigma}}.$$

Next, we compute the value of the formal derivative of $\Lambda(x)$ at X_l^{-1} , i.e.,

$$\Lambda'(X_l^{-1}) := X_l \prod_{\substack{i=1 \\ i \neq l}}^v (1 - X_i X_l^{-1}).$$

According to the Forney formula, the error value at position i_l is given by

$$Y_l := \frac{\Omega(X_l^{-1})}{\Lambda'(X_l^{-1})}.$$

Finally, we subtract the computed error values Y_l from the corresponding positions in the received polynomial $r(x)$ to recover the original codeword $c(x)$.

Distance-Informed Erasure Decoding

To further compress the sizes of the ciphertext and public key while maintaining a reliable **Decoding Failure Rate** (DFR) and proven security, Mito can adopt the *distance-informed erasure decoding* technique introduced by the HARE scheme [WTW⁺26].

Let $\mathcal{C}$ be an $[N, K, D]_q$ linear code. Let $I = \{0, 1, \dots, N-1\}$ be the set of all coordinate indices, and $T \subset I$ be a subset of size $\#T = t$. The projection map

$$\text{pr}_{I \setminus T} : \mathbb{F}_q^N \rightarrow \mathbb{F}_q^{N-t}, (x_0, x_1, \dots, x_{N-1}) \mapsto (x_i)_{i \in I \setminus T}$$

induces an $[N-t, K, D']_q$ -linear code $\mathcal{C}_T = \{\mathbf{x} \in \text{pr}_{I \setminus T}(\mathcal{C})\}$. In particular, if $\mathcal{C}$ is an MDS code, then its parameters satisfy

$$D' = (N-t) - K + 1.$$

Since $\mathcal{C}_T$ can be correctly decoded when the number of errors ν satisfies $2\nu + 1 \leq D'$, we get the correct decoding condition for the *punctured code* $\mathcal{C}_T$ as

$$2\nu + t \leq N - K.$$

In the context of concatenated RSRM decoding, we can model the outer code as a punctured $\text{SRS}(N_{\text{out}}, K_{\text{out}})_q$ code by identifying the index set T with the *erasure positions*. Specifically, if T coincides with the actual error locations introduced during channel transmission, under a constant redundancy budget of $2\sigma = N_{\text{out}} - K_{\text{out}}$, the maximum number of corrupted symbols for the outer code is boosted from σ to $\sigma + \lfloor t/2 \rfloor$.

To achieve this, we rely on the *modified RM decoding* obtained via the FHT. By incorporating a decision mechanism governed by an auxiliary threshold α prior to outputting the ML result via the FHT, we can erase severely distorted symbols decoded by the RM code. The procedure for this modified RM decoding is detailed in [Algorithm 4](#).

Next, to mathematically incorporate the t erasures declared by the inner RM decoder, let the received polynomial be denoted by

$$r(x) = c(x) + e(x) + w(x),$$

where $c(x)$ is the original codeword, $e(x)$ is the unknown error polynomial with ν non-zero terms, and $w(x)$ is the erasure polynomial with t non-zero terms. Suppose the ν errors occur at positions $i_1, \dots, i_\nu$, and the t erasures occur at known positions $l_1, \dots, l_t$. We define the *error locators* $X_j = \alpha^{i_j}$ and the *erasure locators* $Z_m = \alpha^{l_m}$.

In addition to the *error locator polynomial*

$$\Lambda(x) = \prod_{l=1}^{\nu} (1 - X_l x) = 1 + \sum_{l=1}^{\nu} \Lambda_l x^l,$$

we define the *erasure locator polynomial* as

$$\Gamma(x) = \prod_{m=1}^t (1 - Z_m x) = 1 + \sum_{m=1}^t \Gamma_m x^m.$$

Algorithm 4 Modified FHT Decoder for t -RM(1, m)

Input: The received vector y , LLR matrix $(\text{LLR}(y_{z,j}) : z \in \mathbb{F}_2^m, j \in [t])$, and a distance threshold α .

Output: A message $m \in \mathbb{F}_2^{m+1}$ or an erasure symbol $\perp$.

```

1:  $L_z \leftarrow \sum_{j=1}^t \text{LLR}(y_{z,j})$  for all  $z \in \mathbb{F}_2^m$  /* Aggregate LLRs over  $t$  duplications */
/* Compute the Fast Hadamard Transform */
2:  $(\widehat{L}(u) : u \in \mathbb{F}_2^m) \leftarrow \text{FHT}(L_z : z \in \mathbb{F}_2^m)$ 
/* Find the indices of the maximum and second maximum absolute correlations */
3:  $u_1 \leftarrow \arg \max_{u \in \mathbb{F}_2^m} |\widehat{L}(u)|, u_2 \leftarrow \arg \max_{u \in \mathbb{F}_2^m \setminus \{u_1\}} |\widehat{L}(u)|$ 
/* Recover the top two messages based on the signs of their peaks */
4: for  $i = 1, 2$  do
5:   if  $\widehat{L}(u_i) > 0$  then
6:      $m_i \leftarrow (0, u_i)$ 
7:   else
8:      $m_i \leftarrow (1, u_i)$ 
/* Compute Hamming distances to the corresponding codewords */
9:  $d_1 \leftarrow d_H(y, m_1 G_{t\text{-RM}(1,m)}), d_2 \leftarrow d_H(y, m_2 G_{t\text{-RM}(1,m)})$ 
/* Threshold-based confidence check */
10: if  $d_2 - d_1 > \alpha$  then
11:   return  $m_1$ 
12: else
13:   return  $\perp$ 

```

By definition, the inverses of the locators are roots of their respective polynomials, yielding

$$\Lambda(X_j^{-1}) = \sum_{l=0}^v \Lambda_l X_j^{-l} = 0, \quad \Gamma(Z_m^{-1}) = \sum_{l=0}^t \Gamma_l Z_m^{-l} = 0$$

for $j \in \{1, \dots, v\}$ and $m \in \{1, \dots, t\}$.

The primary modification to the classical RS decoding procedure lies in **Step 2**. To explicitly annihilate the known erasure components before executing the BM algorithm, we calculate the **modified syndrome polynomial** $T(x)$, defined as

$$T(x) \equiv (S(x)\Gamma(x)) \pmod{x^{2\sigma}},$$

where $S(x)$ is the classical syndrome polynomial with coefficients

$$S_k = \sum_{i=1}^v Y_i X_i^k + \sum_{m=1}^t W_m Z_m^k.$$

Here, Y_i and W_m represent the error value and erasure value at X_i and Z_m , respectively. Let the modified syndromes be denoted by T_j , such that

$$T(x) = \sum_{j=1}^{2\sigma} T_j x^{j-1}.$$

Then, for $j = t + 1, \dots, 2\sigma$, the computation of T_j proceeds as

$$T_j = \sum_{l=0}^t \Gamma_l S_{j-l}$$

$$\begin{aligned}
&= \sum_{l=0}^t \Gamma_l \left(\sum_{i=1}^v Y_i X_i^{j-l} + \sum_{m=1}^t W_m Z_m^{j-l} \right) \\
&= \sum_{i=1}^v Y_i X_i^j \underbrace{\left(\sum_{l=0}^t \Gamma_l X_i^{-l} \right)}_{\Gamma(X_i^{-1})} + \sum_{m=1}^t W_m Z_m^j \underbrace{\left(\sum_{l=0}^t \Gamma_l Z_m^{-l} \right)}_{\Gamma(Z_m^{-1})=0} \\
&= \sum_{i=1}^v (Y_i \Gamma(X_i^{-1})) X_i^j.
\end{aligned}$$

According to the theory of linear recurring sequences, a sequence formed by the sum of v geometric progressions is annihilated by the polynomial whose roots are the inverses of the progression bases. Hence, this annihilating polynomial is precisely the error locator polynomial $\Lambda(x)$. Therefore, the modified syndromes T_j and the coefficients Λ_i satisfy the linear recurrence relation

$$T_j + \sum_{i=1}^v \Lambda_i T_{j-i} = 0 \quad \text{for } j = v+1, \dots, 2\sigma - t.$$

Consequently, we can determine the coefficients of $\Lambda(x)$ by feeding the modified syndromes $T_{t+1}, T_{t+2}, \dots, T_{2\sigma}$ into the BM algorithm.

Notice that since the decoding capacity is bounded by $2v \leq 2\sigma - t$, the maximal degree of the LFSR is $\lfloor (2\sigma - t)/2 \rfloor$. Therefore, [Algorithm 3](#) now requires exactly $2\sigma - t$ iterations rather than 2σ to find the minimal polynomial. After retrieving $\Lambda(x)$, we find its roots to compute all the error positions X_j . From this point onward, since both the error and erasure positions are identified, they are treated equivalently, and the subsequent error value computation follows the classical [Step 3](#).

Duplicated RM Codes and Shortened RS Codes Used in Mito.

In what follows, we denote the base scheme utilizing [classical RSRM concatenated codes](#) for error correction as Mito, and the variant employing [distance-informed erasure decoding](#) as Mito-E. The duplicated RM codes used in each parameter set are listed in [Table 2.1](#).

Because concatenated codes require consistency between the inner and outer codes, all shortened RS codes utilized in our scheme are defined over the base field $\mathbb{F}_{2^8} = \mathbb{F}_2[x]/\langle x^8 + x^4 + x^3 + x^2 + 1 \rangle$. For computational convenience, we fix the root α of the extension polynomial $f(x) = x^8 + x^4 + x^3 + x^2 + 1$ as the [primitive element](#) that generates the multiplicative group $\mathbb{F}_{2^8}^\times$. Since the ambient code length $N = q - 1 = 255$ is fixed, the specific RS codes we employ are uniquely determined by their respective generator polynomials. We list the exact parameters of the shortened RS codes used in our parameter sets in [Table 2.2](#).

[Table 2.3](#) details the generator polynomials corresponding to our different parameter sets. Let a generator polynomial of degree 2σ be represented as

$$g(x) = \prod_{i=1}^{2\sigma} (x - \alpha^i) = g_0 + g_1 x + g_2 x^2 + \dots + g_{2\sigma} x^{2\sigma}.$$

To ensure clarity for implementation, the coefficients are listed in ascending order of their corresponding polynomial degrees, i.e., the sequence $(g_0, g_1, \dots, g_{2\sigma})$. Each coefficient is presented in hexadecimal format with a 0x prefix.

For example, if we consider g_{12} as a polynomial in $\mathbb{B}[x]$, its explicit expression is given by

$$\begin{aligned}
g_{12}(x) = & 193 + 108x + 199x^2 + 208x^3 + 173x^4 + 79x^5 + 45x^6 + 133x^7 + 251x^8 \\
& + 125x^9 + 44x^{10} + 167x^{11} + 198x^{12} + 150x^{13} + 174x^{14} + 252x^{15} + 218x^{16} \\
& + 8x^{17} + 197x^{18} + 195x^{19} + 20x^{20} + 33x^{21} + 197x^{22} + 244x^{23} + x^{24}.
\end{aligned}$$

Table 2.1. Duplicated Reed-Muller codes used in Mito.

Instance	λ	Original RM Code	Multiplicity	Duplicated RM Code
Mito ₁	128	[128, 8, 64]	3	[384, 8, 192]
Mito ₁ -E	128	[128, 8, 64]	3	[384, 8, 192]
Mito ₂ -E	128	[128, 8, 64]	2	[256, 8, 128]
Mito ₁	256	[128, 8, 64]	3	[384, 8, 192]
Mito ₁ -E	256	[128, 8, 64]	3	[384, 8, 192]
Mito ₂ -E	256	[128, 8, 64]	2	[256, 8, 128]
Mito ₁	512	[128, 8, 64]	5	[640, 8, 320]
Mito ₁ -E	512	[128, 8, 64]	5	[640, 8, 320]
Mito ₂ -E	512	[128, 8, 64]	3	[384, 8, 192]

Table 2.2. Shortened RS codes used in Mito.

Instance	λ	Generator	Original RS Code	s	Shortened RS Code
Mito ₁	128	g_{12}	[255, 231, 25] _{2⁸}	215	[40, 16, 25] _{2⁸}
Mito ₁ -E	128	g_{11}	[255, 233, 23] _{2⁸}	217	[38, 16, 23] _{2⁸}
Mito ₂ -E	128	g_{12}	[255, 231, 25] _{2⁸}	215	[40, 16, 25] _{2⁸}
Mito ₁	256	g_{28}	[255, 199, 57] _{2⁸}	167	[88, 32, 57] _{2⁸}
Mito ₁ -E	256	g_{26}	[255, 203, 53] _{2⁸}	171	[84, 32, 53] _{2⁸}
Mito ₂ -E	256	g_{26}	[255, 203, 53] _{2⁸}	171	[84, 32, 53] _{2⁸}
Mito ₁	512	g_{51}	[255, 153, 103] _{2⁸}	89	[166, 64, 103] _{2⁸}
Mito ₁ -E	512	g_{48}	[255, 159, 97] _{2⁸}	95	[160, 64, 97] _{2⁸}
Mito ₂ -E	512	g_{53}	[255, 149, 107] _{2⁸}	85	[170, 64, 107] _{2⁸}

Table 2.3. Generator Polynomials for Shortened RS Codes in Mito

λ	Poly.	Deg.	Coefficients in Hexadecimal
128	g_{12}	24	0xC1, 0x6C, 0xC7, 0xD0, 0xAD, 0x4F, 0x2D, 0x85, 0xFB, 0x7D, 0x2C, 0xA7, 0xC6, 0x96, 0xAE, 0xFC, 0xDA, 0x08, 0xC5, 0xC3, 0x14, 0x21, 0xC5, 0xF4, 0x01
	g_{11}	22	0x47, 0x6E, 0x33, 0x4B, 0x49, 0xCB, 0xD9, 0x1F, 0x6E, 0x75, 0xB1, 0xB2, 0xA1, 0xF4, 0xED, 0x75, 0xCB, 0x43, 0xCF, 0x6C, 0xF6, 0xB2, 0x01
256	g_{28}	56	0x61, 0xF8, 0x89, 0x20, 0x8E, 0xA7, 0x38, 0xD4, 0xFA, 0x89, 0x68, 0x51, 0xC2, 0x42, 0x1B, 0x19, 0xC6, 0x84, 0x8A, 0x75, 0x76, 0xC1, 0xDE, 0x09, 0x53, 0x51, 0x8F, 0xB2, 0xD7, 0x39, 0x91, 0x4A, 0xF3, 0x19, 0xF4, 0xE4, 0xFF, 0x2D, 0xBF, 0xC9, 0xE9, 0x9B, 0xEF, 0x8C, 0xFF, 0x10, 0x22, 0xE8, 0x97, 0x93, 0x8A, 0xE5, 0xD1, 0x67, 0xEC, 0x68, 0x01
	g_{26}	52	0x88, 0x7E, 0x8C, 0x27, 0xE7, 0x52, 0xE8, 0xE9, 0xC0, 0xD3, 0x42, 0xCC, 0xF8, 0x6F, 0x73, 0x98, 0x13, 0xA9, 0x29, 0xD5, 0xAF, 0xDF, 0x97, 0x56, 0xED, 0x06, 0xE6, 0x3D, 0xE8, 0xB9, 0x7E, 0xFD, 0x4E, 0xD4, 0xF3, 0xFC, 0x36, 0x0E, 0x27, 0x9E, 0x42, 0xAF, 0x5D, 0x38, 0xAF, 0x08, 0x48, 0xA0, 0xAE, 0xE1, 0x14, 0xED, 0x01
512	g_{51}	102	0x92, 0x83, 0x54, 0xF0, 0x52, 0x0A, 0x51, 0xD8, 0xB8, 0x3B, 0xC6, 0x22, 0x5B, 0xAC, 0x67, 0xC6, 0x86, 0xDC, 0x9D, 0xAA, 0xBB, 0x05, 0xA4, 0xD4, 0x9E, 0x26, 0x09, 0xE4, 0x7F, 0xAE, 0x9D, 0x67, 0x1F, 0x5B, 0x98, 0x57, 0x1F, 0x92, 0x61, 0x45, 0x17, 0x01, 0x1A, 0x5D, 0x93, 0x9E, 0x06, 0x01, 0x74, 0x8A, 0xD2, 0x91, 0xB2, 0x4C, 0xB2, 0x43, 0x75, 0x30, 0x63, 0xAD, 0x07, 0xE9, 0x4C, 0x20, 0x04, 0x58, 0x43, 0xBC, 0x0A, 0xC9, 0xC8, 0xDF, 0xC6, 0x56, 0x87, 0xFB, 0x07, 0x2A, 0xD1, 0x9B, 0x8A, 0xB4, 0xA7, 0xD7, 0xE0, 0x01, 0x5F, 0x06, 0x97, 0x5C, 0x72, 0x6F, 0x75, 0x17, 0x94, 0x50, 0x63, 0xB7, 0x49, 0xEB, 0xC8, 0x8D, 0x01
	g_{48}	96	0x61, 0xC8, 0x90, 0x81, 0x3C, 0x76, 0x46, 0x63, 0x3C, 0xEE, 0x88, 0x13, 0x90, 0x6B, 0xE1, 0x27, 0x8A, 0xB0, 0xAF, 0x42, 0x1B, 0xB4, 0xC6, 0xAD, 0xD1, 0x18, 0xC5, 0x87, 0x4F, 0x73, 0xB0, 0xD1, 0x09, 0xF9, 0x5E, 0x61, 0x45, 0x3E, 0x51, 0xAA, 0xB6, 0xE8, 0x8E, 0x9D, 0x3F, 0x12, 0x5A, 0x33, 0xB4, 0x18, 0xFD, 0xA5, 0x27, 0xBA, 0x62, 0xAF, 0xD3, 0x77, 0x08, 0xC2, 0x3C, 0xFD, 0xB0, 0x4E, 0x38, 0xB5, 0x06, 0x22, 0xCB, 0x4C, 0x30, 0x5B, 0x85, 0xEC, 0x82, 0x53, 0xD3, 0x5C, 0x83, 0x6A, 0xAB, 0x29, 0x34, 0x10, 0x05, 0x6C, 0xB0, 0xB1, 0xEA, 0xE8, 0x50, 0x60, 0xA8, 0xD3, 0x11, 0x90, 0x01
	g_{53}	106	0x6F, 0x12, 0xDE, 0x98, 0x54, 0x97, 0xDA, 0x31, 0x55, 0xBC, 0x54, 0x28, 0x80, 0x4C, 0x81, 0x40, 0xFB, 0x21, 0xCD, 0x80, 0xC1, 0x41, 0xD4, 0x5D, 0xC0, 0x80, 0x54, 0xCC, 0x30, 0x8B, 0xC2, 0xD2, 0x52, 0xBF, 0x57, 0x03, 0xB6, 0x63, 0x6A, 0x21, 0xE0, 0x74, 0x7F, 0x90, 0x90, 0x10, 0x02, 0xFA, 0xEF, 0x2C, 0x5B, 0xF8, 0x51, 0x0F, 0xD3, 0x09, 0x36, 0x05, 0xF1, 0xEF, 0xBB, 0x20, 0x91, 0x6F, 0xDB, 0x37, 0x02, 0x7F, 0xD7, 0xAC, 0xCB, 0xD2, 0x42, 0xC8, 0xDE, 0xC8, 0x01, 0xE4, 0x56, 0x73, 0x55, 0xD2, 0xBC, 0x7D, 0xD1, 0x35, 0xD3, 0x6B, 0x32, 0x5B, 0xCE, 0x11, 0xB4, 0x5A, 0xBC, 0x59, 0xA4, 0xB8, 0x51, 0x96, 0x0D, 0xA7, 0xFE, 0x01, 0xEA, 0xEF, 0x26, 0x01

3 Algorithm Specification

3.1 Variables and Functions

In this section, we summarize the system parameters, variables, and auxiliary functions used in the Mito scheme.

3.1.1 Variables

Table 3.1. System Parameters parm

Notation	Domain	Description
λ	$\{128, 256, 512\}$	Security parameter.
λ_{seed}	$\{64\}$	Length of the seed in bytes.
λ_{salt}	$\{32\}$	Length of the salt in bytes.
ℓ	$\{1, 2\}$	Adic number.
k	$\{16, 32, 64\}$	Length of the message in Mito.PKE in bytes.
$\mathbf{w}_x$	$\mathbb{N}^{2^\ell}$	Target Hamming weights for $\mathbf{x}$.
$\mathbf{w}_y$	$\mathbb{N}^{2^\ell}$	Target Hamming weights for $\mathbf{y}$.
$\mathbf{w}_r$	$\mathbb{N}^{2^\ell}$	Target Hamming weights for $\mathbf{r}$.
$\mathbf{w}_t$	$\mathbb{N}^{2^\ell}$	Target Hamming weights for $\mathbf{t}$.
w_e	$\mathbb{N}$	Hamming weight of the error vector $\mathbf{e}$.
E	$\{0, 1\}$	Switch for distance-informed erasure decoding.

Table 3.2. Coding Parameters

Notation	Domain	Description
n	$\mathbb{N}$	Dimension of the base ring $\mathcal{R}_0$.
n'	$\mathbb{N}$	Length of the concatenated code.
n_{in}	$\mathbb{N}$	Length of the inner code.
N_{out}	$\mathbb{N}$	Length of the outer code.
k_{in}	$\mathbb{N}$	Dimension of the inner code.
K_{out}	$\mathbb{N}$	Dimension of the outer code.
d_{in}	$\mathbb{N}$	Minimum distance of the inner code.
D_{out}	$\mathbb{N}$	Minimum distance of the outer code.

Table 3.3. Algorithm variables

Notation	Domain	Description
seed_{PKE}	$\mathbb{B}^{\lambda_{\text{seed}}}$	Random seed for Mito.PKE.
seed_{KEM}	$\mathbb{B}^{\lambda_{\text{seed}}}$	Random seed for Mito.KEM.
seed_{sk}	$\mathbb{B}^{\lambda_{\text{seed}}}$	Random seed to generate sk.
seed_{pk}	$\mathbb{B}^{\lambda_{\text{seed}}}$	Random seed to generate pk.
θ, σ	$\mathbb{B}^{\lambda_{\text{seed}}}$	Cryptographic randomness.
salt	$\mathbb{B}^{\lambda_{\text{salt}}}$	Salt value.
state_{θ}	$\mathbb{F}_2^{\infty}$	Internal state of hash function induced by θ .
state_{pk}	$\mathbb{F}_2^{\infty}$	Internal state of hash function induced by pk.
$\text{state}_{\mathbf{w}}$	$\mathbb{F}_2^{\infty}$	Internal state of hash function induced by $\mathbf{w}$.
$\text{state}_{\text{KEM}}$	$\mathbb{F}_2^{\infty}$	Internal state of hash function induced by seed_{KEM} .
K	$\mathbb{B}^{\lambda_{\text{seed}}}$	Shared secret for Mito.KEM.
$\bar{K}$	$\mathbb{B}^{\lambda_{\text{seed}}}$	Masked secret for Mito.KEM.
K'	$\mathbb{B}^{\lambda_{\text{seed}}}$	Output secret for Mito.KEM.
sk	$\mathbb{B}^{\lambda_{\text{seed}}}$	Secret key for Mito.PKE.
pk	$\mathbb{B}^{\lambda_{\text{seed}}} \times (\mathbb{F}_2^n)^{2^\ell}$	Public key for Mito.PKE.
ek	$\mathbb{B}^{\lambda_{\text{seed}}} \times (\mathbb{F}_2^n)^{2^\ell}$	Encapsulation key for Mito.KEM.
dk	$\mathbb{B}^{3\lambda_{\text{seed}}} \times (\mathbb{F}_2^n)^{2^\ell}$	Decapsulation key for Mito.KEM.

Table 3.4. Intermediate Variables

Notation	Domain	Description
$\mathbf{m}$	$\mathbb{B}^k$	Message for Mito.PKE.Encrypt.
$\mathbf{m}'$	$\mathbb{B}^k$	Received message for Mito.PKE.Decrypt.
$\mathbf{c}$	$(\mathbb{F}_2^n)^{2^\ell} \times \mathbb{F}_2^{n'}$	Ciphertext for Mito.PKE.
$\mathbf{c}'$	$(\mathbb{F}_2^n)^{2^\ell} \times \mathbb{F}_2^{n'}$	Ciphertext for check.
$\mathbf{ct}$	$(\mathbb{F}_2^n)^{2^\ell} \times \mathbb{F}_2^{n'} \times \mathbb{B}^{\lambda_{\text{salt}}}$	Ciphertext for Mito.KEM.
$\mathbf{h}$	$(\mathbb{F}_2^n)^{2^\ell}$	Random vector.
$\mathbf{x}, \mathbf{y}$	$(\mathbb{F}_2^n)^{2^\ell}$	Vectors used to generate the public key.
$\mathbf{s}$	$(\mathbb{F}_2^n)^{2^\ell}$	Syndrome generated during Mito.PKE.KeyGen.
$\mathbf{r}, \mathbf{t}$	$(\mathbb{F}_2^n)^{2^\ell}$	Randomness vectors for encryption.
$\mathbf{e}$	$\mathbb{F}_2^n$	Error vector introduced in encryption.
$\mathbf{u}$	$(\mathbb{F}_2^n)^{2^\ell}$	Syndrome generated during Mito.PKE.Encrypt.
$\mathbf{v}$	$\mathbb{F}_2^{n'}$	Random vectors used to hide the message.

3.1.2 Functions

System Function Setup(1^λ). On input a security parameter λ , this algorithm generates and outputs the public system parameters parm

$$\text{parm} := (\lambda_{\text{seed}}, \lambda_{\text{salt}}, \ell, k, \mathbf{w}_x, \mathbf{w}_y, \mathbf{w}_r, \mathbf{w}_t, w_e, E).$$

Cryptographic Hash and XOF Primitives. Given a base hash function H and an extendable-output function XOF achieving a target security level λ , we construct a set of mutually independent cryptographic primitives. This independence is rigorously instantiated via domain separation and by tailoring the input and output spaces. We define the following specialized functions:

- $H_{\text{seed}}: \mathbb{B}^{\lambda_{\text{seed}}} \rightarrow \mathbb{B}^{2\lambda_{\text{seed}}}$: A hash function expanding a fixed-length seed to twice its length.
- $H_{\text{comp}}: \mathbb{F}_2^\infty \rightarrow \mathbb{B}^{2\lambda_{\text{seed}}}$: A hash function utilized for compressing arbitrary-length inputs, primarily to hash the encapsulation key ek .
- $H_{\text{gen}}: \mathbb{F}_2^\infty \rightarrow \mathbb{B}^{\lambda_{\text{seed}}}$: A hash function employed for key derivation or extracting structured randomness.
- $H_{\text{mask}}: \mathbb{F}_2^\infty \rightarrow \mathbb{B}^{\lambda_{\text{seed}}}$: A hash function designated to generate masking values.
- $XOF_{\text{Init}}: \mathbb{F}_2^\infty \rightarrow \mathbb{F}_2^\infty$: An extendable-output function utilized to derive pseudorandom bit-streams for initial vectors.
- $XOF_{\text{Bytes}}: \mathbb{F}_2^\infty \rightarrow \mathbb{B}^\infty$: An extendable-output function tailored to generate arbitrary-length byte strings.

Sampling Algorithms

- $\text{Sample}_{\text{Unif}}: \mathbb{B}^{\lambda_{\text{seed}}} \rightarrow \mathbb{F}_2^n$: Generates a uniformly distributed random vector in $\mathbb{F}_2^n$ by expanding the provided seed using the XOF_{Bytes} function.
- $\text{Sample}_{\text{DisWt}}: \mathbb{B}^{\lambda_{\text{seed}}} \times \mathbb{N}^{2^\ell} \rightarrow (\mathbb{F}_2^n)^{2^\ell}$: To sample a tuple of vectors adhering to a specific weight distribution

$$\mathbf{w} = (w[0], \dots, w[2^\ell - 1]) \in \mathbb{N}^{2^\ell},$$

this algorithm leverages the $\text{SampleFixedWeightVect}$ function introduced in [GAMA⁺25, Section 3.2]. Specifically, $\text{Sample}_{\text{DisWt}}$ first utilizes the seed to sample a single vector $v \in \mathbb{F}_2^n$ with a total Hamming weight of

$$W = \sum_{i=0}^{2^\ell-1} w[i].$$

Based on the sequential order in which the random non-zero positions are generated, it partitions the support of v into 2^ℓ disjoint subsets, where the i -th subset contains exactly $w[i]$ indices. These subsets define a sequence of 2^ℓ sub-vectors $v_0, \dots, v_{2^\ell-1} \in \mathbb{F}_2^n$ such that

$$v = \sum_{i=0}^{2^\ell-1} v_i.$$

Finally, the algorithm outputs this ordered tuple of vectors.

Algebraic Operations

- $\text{Encode}: \mathbb{F}_2^k \rightarrow \mathbb{F}_2^{n'}$: The encoding procedure for the concatenated RSRM code, realized by composing the algorithm described in Algorithm 2 with the generator matrix of the t -RM(1,7) code.
- $\text{Decode}: \{0, 1\} \times \mathbb{F}_2^{n'} \rightarrow \mathbb{F}_2^k$: The decoding procedure for the concatenated RSRM code. The system parameter E determines whether to employ the distance-informed erasure decoding strategy.

- $\text{Trun}: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^{n'}$: Truncates the rightmost $(n - n')$ bits of an input vector.
- $\text{Pr}: (\mathbb{F}_2^n)^{2^\ell} \rightarrow \mathbb{F}_2^n$: A projection map that extracts the first component v_0 from a sequence of vectors $(v_0, \dots, v_{2^\ell-1})$.
- $\text{Adic}(\cdot): (\mathbb{F}_2^n)^{2^\ell} \rightarrow \mathbb{F}_2^{2^\ell n \times 2^\ell n}$: Maps a sequence of vectors to its corresponding adic matrix. It is strictly required that throughout the Mito scheme, all vector operations, including matrix-vector multiplications, are intrinsically induced by polynomial multiplication $*$ over the base ring $\mathcal{R}_0$. The recursive Karatsuba strategy is employed to minimize the multiplication complexity over $\mathcal{R}_0$.

Example 3.1.1. The following outlines the explicit computation paths for the matrix-vector multiplication $\mathbf{z} = \mathbf{y} * \text{Adic}(\mathbf{h})$ in the 2-adic and 2^2 -adic cases.

Case 1: $\ell = 1$. Let $\mathbf{y} = (y_0, y_1)$ and $\mathbf{h} = (h_0, h_1)$. By exploiting the adic symmetry, we compute this using exactly 3 multiplications:

$$\begin{aligned} m_0 &= y_0 * h_0, \\ m_1 &= y_1 * h_1, \\ m_2 &= (y_0 + y_1) * (h_0 + h_1). \end{aligned}$$

The resultant vector $\mathbf{z} = (z_0, z_1)$ is reconstructed purely via addition

$$z_0 = m_0 + m_1, \quad z_1 = m_0 + m_1 + m_2.$$

Case 2: $\ell = 2$. Let $\mathbf{y} = (y_0, y_1, y_2, y_3)$ and $\mathbf{h} = (h_0, h_1, h_2, h_3)$. Applying the above strategy recursively reduces the naive 16 operations to $3^2 = 9$ base multiplications over $\mathcal{R}_0$:

$$\begin{aligned} m_0 &= y_0 * h_0, \\ m_1 &= y_1 * h_1, \\ m_2 &= (y_0 + y_1) * (h_0 + h_1), \\ m_3 &= y_2 * h_2, \\ m_4 &= y_3 * h_3, \\ m_5 &= (y_2 + y_3) * (h_2 + h_3), \\ m_6 &= (y_0 + y_2) * (h_0 + h_2), \\ m_7 &= (y_1 + y_3) * (h_1 + h_3), \\ m_8 &= (y_0 + y_1 + y_2 + y_3) * (h_0 + h_1 + h_2 + h_3). \end{aligned}$$

The final vector $\mathbf{z} = (z_0, z_1, z_2, z_3)$ is then linearly assembled from these 9 intermediates

$$\begin{aligned} z_0 &= m_0 + m_1 + m_3 + m_4, \\ z_1 &= m_0 + m_1 + m_2 + m_3 + m_4 + m_5, \\ z_2 &= z_0 + m_6 + m_7, \\ z_3 &= z_1 + m_6 + m_7 + m_8. \end{aligned}$$

△

3.2 The Mito PKE Scheme

Algorithm 5 Mito.PKE.KeyGen(seed_{PKE}, parm)

Input: Random seed seed_{PKE}, system parameters parm.

Output: Public key pk, secret key sk.

```

1: (seedsk, seedpk) ← Hseed(seedPKE) /* Expand the master seed to generate seeds for the secret and public keys */
2: statesk ← XOFInit(seedsk) /* Initialize the XOF internal state using the secret key seed */
3: (wy[0], ..., wy[2ℓ - 1]) ← wy /* Sample sparse secret vectors while maintaining the XOF state */
4: statew ← ∅
5: for i = 0, ..., 2ℓ - 1 do
6:   (statesk, statew, yi) ← SampleDisWt(statesk, statew, wy[i])
7: y ← (y0, ..., y2ℓ-1)
8: (wx[0], ..., wx[2ℓ - 1]) ← wx
9: statew ← ∅
10: for i = 0, ..., 2ℓ - 1 do
11:   (statesk, statew, xi) ← SampleDisWt(statesk, statew, wx[i])
12: x ← (x0, ..., x2ℓ-1) /* Initialize the XOF internal state using the public key seed */
13: statepk ← XOFInit(seedpk) /* Sample uniformly random vectors to form the public sequence */
14: for i = 0, ..., 2ℓ - 1 do
15:   (statepk, hi) ← SampleUnif(statepk)
16: h ← (h0, ..., h2ℓ-1) /* Compute the public syndrome s */
17: s ← x + y * Adic(h) /* Pack the public and secret keys */
18: sk ← seedsk
19: pk ← (seedpk, s)
20: return (pk, sk)

```

Algorithm 6 Mito.PKE.Encrypt(pk, **m**, parm, θ)

Input: Public key pk, message **m**, randomness θ , system parameters parm.

Output: Ciphertext **c** = (**u**, **v**).

```
1: (seedpk, s)  $\leftarrow$  pk
2: statepk  $\leftarrow$  XOFInit(seedpk)
3: for  $i = 0, \dots, 2^\ell - 1$  do
4:   (statepk,  $h_i$ )  $\leftarrow$  SampleUnif(statepk)
5: h  $\leftarrow$  ( $h_0, \dots, h_{2^\ell-1}$ )
6: state $_\theta$   $\leftarrow$  XOFInit( $\theta$ )
7: ( $w_r[0], \dots, w_r[2^\ell - 1]$ )  $\leftarrow$  wr
8: statew  $\leftarrow$   $\emptyset$ 
9: for  $i = 0, \dots, 2^\ell - 1$  do
10:  (state $_\theta$ , statew,  $r_i$ )  $\leftarrow$  SampleDisWt(state $_\theta$ , statew,  $w_r[i]$ )
11: r  $\leftarrow$  ( $r_0, \dots, r_{2^\ell-1}$ )
12: ( $w_t[0], \dots, w_t[2^\ell - 1]$ )  $\leftarrow$  wt
13: statew  $\leftarrow$   $\emptyset$ 
14: for  $i = 0, \dots, 2^\ell - 1$  do
15:  (state $_\theta$ , statew,  $t_i$ )  $\leftarrow$  SampleDisWt(state $_\theta$ , statew,  $w_t[i]$ )
16: t  $\leftarrow$  ( $t_0, \dots, t_{2^\ell-1}$ )
17: statew  $\leftarrow$   $\emptyset$ 
18: (state $_\theta$ , statew, e)  $\leftarrow$  SampleDisWt(state $_\theta$ , statew,  $w_e$ )
19: u  $\leftarrow$  r + t * Adic(h)
20: v  $\leftarrow$  Encode(m) + Trun(e + Pr(s * Adic(t)))
21: c  $\leftarrow$  (u, v)
22: return c
```

Algorithm 7 Mito.PKE.Decrypt(sk, **c**, parm)

Input: Secret key sk, ciphertext **c**, system parameters parm.

Output: Message **m**.

```
1: seedsk ← sk /* Parse secret key and ciphertext */
2: (u, v) ← c
3: statesk ← XOFInit(seedsk) /* Initialize the XOF internal state using the secret key seed */
4: (wy[0], ..., wy[2ℓ - 1]) ← wy /* Reconstruct the secret vector sequence y */
5: statew ← ∅
6: for i = 0, ..., 2ℓ - 1 do
7:   (statesk, statew, yi) ← SampleDisWt(statesk, statew, wy[i])
8: y ← (y0, ..., y2ℓ-1)
9: m ← Decode(E, v - Trun(Pr(u * Adic(y)))) /* Compute the noisy codeword and decode it */
10: return m
```

3.3 The Mito KEM Scheme

Algorithm 8 Mito.KEM.KeyGen(1^λ)

Input: Security parameter 1^λ .**Output:** Encapsulation key ek, Decapsulation key dk.

```
1: parm  $\leftarrow$  Setup( $1^\lambda$ ) /* Select system parameters based on the targeted security level */
/* Sample the master seed */
2: seedKEM  $\xleftarrow{\$}$   $\mathbb{B}^{\lambda_{\text{seed}}}$ 
/* Derive the PKE seed and the implicit rejection value  $\sigma$  */
3: stateKEM  $\leftarrow$  XOFInit(seedKEM)
4: (stateKEM, seedPKE)  $\leftarrow$  XOFBytes(stateKEM,  $\lambda_{\text{seed}}$ )
5: (stateKEM,  $\sigma$ )  $\leftarrow$  XOFBytes(stateKEM,  $\lambda_{\text{seed}}$ )
/* Invoke the underlying PKE KeyGen */
6: (pk, sk)  $\leftarrow$  Mito.PKE.KeyGen(seedPKE, parm)
/* Form KEM keys, securely embedding the implicit rejection parameter */
7: ek  $\leftarrow$  pk
8: dk  $\leftarrow$  (ek, sk,  $\sigma$ )
9: return (ek, dk)
```

Algorithm 9 Mito.KEM.Encaps(ek, parm)

Input: Encapsulation key ek, system parameters parm.

Output: Shared secret K, ciphertext ct.

/ Sample a uniform message and a salt */*

1: $\mathbf{m} \xleftarrow{\$} \mathbb{B}^k$
2: $\text{salt} \xleftarrow{\$} \mathbb{B}^{\lambda_{\text{salt}}}$

/ Derive the shared key K and the encryption randomness θ */*

3: $(K, \theta) \leftarrow H_{\text{gen}}(H_{\text{comp}}(\text{ek}) \parallel \mathbf{m} \parallel \text{salt})$

/ Encrypt the message using the deterministic PKE encryption */*

4: $\mathbf{c} \leftarrow \text{Mito.PKE.Encrypt}(\text{ek}, \mathbf{m}, \text{parm}, \theta)$

/ Form the final KEM ciphertext */*

5: $\text{ct} \leftarrow (\mathbf{c}, \text{salt})$
6: **return** (K, ct)

Algorithm 10 Mito.KEM.Decaps(dk, ct, parm)

Input: Decapsulation key dk, ciphertext ct, system parameters parm.

Output: Shared secret K' .

```

1:  $(ek, sk, \sigma) \leftarrow dk$ 
2:  $(c, salt) \leftarrow ct$ 
3:  $m' \leftarrow \text{Mito.PKE.Decrypt}(sk, c, parm)$ 
4:  $(K', \theta') \leftarrow H_{\text{gen}}(H_{\text{comp}}(ek) \parallel m' \parallel salt)$ 
5:  $c' \leftarrow \text{Mito.PKE.Encrypt}(ek, m', parm, \theta')$ 
6:  $\bar{K} \leftarrow H_{\text{gen}}(H_{\text{comp}}(ek) \parallel \sigma \parallel ct)$ 
7: if  $m' = \perp$  or  $c' \neq c$  then
8:    $K' \leftarrow \bar{K}$ 
9: return  $K'$ 
```

3.4 System Parameters and Sizes

This section provides three parameter sets, corresponding to Mito₁.PKE, Mito₁-E.PKE, and Mito₂-E.PKE, respectively, and presents the resulting key and ciphertext sizes for the corresponding KEMs.

For each parameter set, we consider both the classical security level λ and the quantum security level λ_Q . The parameters are deliberately chosen such that the minimal work factor of the best-known attacks strictly exceeds the targeted security limits. In practice, these configurations are jointly determined by the computational complexity of ISD algorithms at the respective security levels and the DFR of the underlying PKE (see [Section 4.3](#) for a detailed security analysis).

3.4.1 System Parameter Sets

The baseline scheme, denoted as Mito₁, employs the classical RSRM concatenated code decoding technique for noise elimination. It relies on a 2-adic structure, providing a standard algebraic foundation. The parameter sets are detailed in [Table 3.5](#).

Table 3.5. Parameter sets for classical Mito₁

Inst.	λ	λ_Q	n	n'	n_{in}	N_{out}	$(\mathbf{w}_x, \mathbf{w}_y)$	$(\mathbf{w}_r, \mathbf{w}_t)$	w_e	p^*	DFR
Mito ₁	128	80	15373	15360	384	40	(44, 41, 47, 40)	(42, 45, 44, 45)	85	0.3161	$2^{-173.34}$
Mito ₁	256	128	33827	33792	384	88	(65, 63, 72, 69)	(63, 66, 71, 74)	133	0.3335	$2^{-272.79}$
Mito ₁	512	256	106261	106240	640	166	(127, 126, 143, 131)	(123, 134, 137, 138)	262	0.3671	$2^{-516.22}$

For comparison, Mito₁-E adopts the distance-informed erasure decoding technique while retaining the 2-adic structure. By declaring erasures to aid the outer decoder, this variant achieves greater flexibility in parameter sizes under the same security levels. The parameter sets are detailed in [Table 3.6](#).

Table 3.6. Parameter sets for erasure-enhanced Mito₁-E

Inst.	λ	λ_Q	n	n'	n_{in}	N_{out}	$(\mathbf{w}_x, \mathbf{w}_y)$	$(\mathbf{w}_r, \mathbf{w}_t)$	w_e	p^*	DFR
Mito ₁ -E	128	80	14621	14592	384	38	(44, 41, 47, 40)	(42, 45, 44, 45)	86	0.3256	$2^{-167.99}$
Mito ₁ -E	256	128	32261	32256	384	84	(69, 64, 72, 64)	(63, 73, 66, 72)	132	0.3420	$2^{-265.48}$
Mito ₁ -E	512	256	102437	102400	640	160	(128, 123, 140, 136)	(126, 129, 136, 141)	261	0.3735	$2^{-517.00}$

Designed for high-performance applications, Mito₂-E.PKE utilizes a 2^2 -adic structure¹ alongside the erasure decoding mechanism. This architecture splits the computational workload into multiple branches, supporting parallel processing. Consequently, it is suitable for high-speed cryptographic operations in multi-core environments. The parameter sets are detailed in [Table 3.7](#).

Table 3.7. Parameter sets for multi-core oriented Mito₂-E

Inst.	λ	λ_Q	n	n'	n_{in}	N_{out}	$\mathbf{w}_1$	$\mathbf{w}_2$	w_e	p^*	DFR
Mito ₂ -E	128	80	10253	10240	256	40	(25, 25, 22, 21)	(22, 23, 25, 25)	39	0.2907	$2^{-181.39}$
Mito ₂ -E	256	128	21523	21504	256	84	(38, 38, 33, 33)	(34, 34, 38, 38)	68	0.3079	$2^{-269.97}$
Mito ₂ -E	512	256	65293	65280	384	170	(72, 72, 66, 62)	(65, 66, 70, 73)	135	0.3409	$2^{-547.37}$

¹To constrain the parameter search space and maintain simplicity, we impose the additional requirements that $\mathbf{w}_x = \mathbf{w}_y = \mathbf{w}_1$ and $\mathbf{w}_r = \mathbf{w}_t = \mathbf{w}_2$ for the 2^2 -adic case.

3.4.2 Ciphertext and Key Sizes

The sizes of the keys and ciphertexts depend on the underlying 2^ℓ -adic structure. Specifically, for the 2^ℓ -adic case, the encapsulation key ek corresponds to the public key of the underlying PKE. It consists of a seed of length $8\lambda_{\text{seed}}$ bits which is used to expand the random vector $\mathbf{h}$ and a public syndrome of length $2^\ell n$ bits generated during the PKE.KeyGen step. Therefore, its total size is given by

$$|ek| = 2^\ell \lceil n/8 \rceil + \lambda_{\text{seed}}$$

in bytes.

Although the decapsulation key dk conceptually comprises the encapsulation key ek , the PKE secret key sk , and implicit randomness δ , all these components can be deterministically derived from a single master seed seed_{KEM} . Consequently, its storage size is

$$|dk| = |ek| + 2\lambda_{\text{seed}} + k \text{ or } \lambda_{\text{seed}}$$

in bytes. Note that the PKE ciphertext $\mathbf{c}$ consists of the vector $\mathbf{u}$ and the masked message $\mathbf{v}$, yielding a size of $\lceil 2^\ell n/8 \rceil + \lceil n'/8 \rceil$ bytes. The KEM ciphertext $\mathbf{ct}$ encapsulates $\mathbf{c}$ alongside an additional salt, bringing its total size to

$$|\mathbf{ct}| = 2^\ell \lceil n/8 \rceil + \lceil n'/8 \rceil + \lambda_{\text{salt}}$$

in bytes. Consequently, the total communication bandwidth, defined as the sum of the sizes of ek and $\mathbf{ct}$, is given by

$$2^{\ell+1} \lceil n/8 \rceil + \lceil n'/8 \rceil + \lambda_{\text{seed}} + \lambda_{\text{salt}}.$$

in bytes.

Finally, the shared session key K has a fixed size of $8k = 64$ bytes. The lengths of the seed and salt are set to $\lambda_{\text{seed}} = 64$ bytes and $\lambda_{\text{salt}} = 32$ bytes, respectively. The resulting key and ciphertext sizes for all instances of the Mito suite, calculated based on the formulas presented above, are summarized in [Table 3.8](#). All sizes are reported in bytes.

Table 3.8. Key and Ciphertext Sizes of the Mito Suite

Scheme	ℓ	λ	$ ek $ (B)	$ dk $ (B)	$ \mathbf{ct} $ (B)	Bandwidth (B)
Mito ₁	1	128	3,908	4,052	5,796	9,704
		256	8,522	8,682	12,714	21,236
		512	26,630	26,822	39,878	66,508
Mito ₁ -E	1	128	3,720	3,864	5,512	9,232
		256	8,130	8,290	12,130	20,260
		512	25,674	25,866	38,442	64,116
Mito ₂ -E	2	128	5,192	5,336	6,440	11,632
		256	10,828	10,988	13,484	24,312
		512	32,712	32,904	40,840	73,552

3.4.3 Performance Benchmark

To validate the practical efficiency of the Mito suite, we provide the timing measurements of a self-implemented reference code² on a commodity desktop platform. The detailed specifications of the test environment are listed in [Table 3.9](#).

²The reference implementation is written in standard C and compiled with Microsoft Visual Studio 2022. All measurements are conducted on a single core without any SIMD instruction-set extensions.

Table 3.9. Test Platform Specifications

Component	Specification
CPU	Intel(R) Core(TM) i3-6100 @ 3.70 GHz
Memory	16 GB
Operating System	Windows 7 (64-bit)
Compiler	Microsoft Visual Studio 2022 (Standard C)
SIMD Extensions	None
Multi-threading	None (single-core)

The performance of each Mito instance is measured across its three primary operations: KeyGen, Encaps, and Decaps. For each operation, we report both the throughput in operations per second (ops/s) and the corresponding CPU kilo-cycles (10^3 cycles), as summarized in [Table 3.10](#).

Table 3.10. Performance of the Mito Suite on the Reference Platform

Instance	λ	KeyGen		Encaps		Decaps	
		ops/s	kcycles	ops/s	kcycles	ops/s	kcycles
Mito ₁	128	178.4	20,737	108.8	34,012	77.3	47,876
Mito ₁	256	58.9	62,857	34.9	106,104	25.0	148,051
Mito ₁	512	9.4	394,744	5.6	656,895	4.0	922,398
Mito ₁ -E	128	197.7	18,717	116.1	31,862	83.9	44,118
Mito ₁ -E	256	55.7	66,383	33.7	109,894	24.0	154,273
Mito ₁ -E	512	9.8	377,949	5.9	630,648	4.2	885,427
Mito ₂ -E	128	130.7	28,303	90.8	40,761	68.7	53,863
Mito ₂ -E	256	40.2	91,958	28.0	132,058	21.2	174,464
Mito ₂ -E	512	6.3	585,294	4.4	845,016	3.3	1,105,924

Comparison with HQC. To contextualize the efficiency of Mito, we benchmark the NIST-standardized HQC scheme [[GAMA⁺25](#)] on the identical platform. [Table 3.11](#) reports the reference sizes of the HQC KEM at each security level, while [Table 3.12](#) presents the corresponding timing measurements.

Table 3.11. Key and Ciphertext Sizes of the HQC KEM

Instance	Level	pk (B)	sk (B)	K (B)	ct (B)
HQC	1	2,241	2,321	32	4,433
HQC	3	4,514	4,602	32	8,978
HQC	5	7,237	7,333	32	14,421

Table 3.12. Performance of HQC on the Reference Platform

Instance	Level	KeyGen		Encaps		Decaps	
		ops/s	kcycles	ops/s	kcycles	ops/s	kcycles
HQC	1	485.4	7,622	254.9	14,516	168.1	22,014
HQC	3	165.0	22,428	85.9	43,074	56.3	65,682
HQC	5	68.7	53,867	34.6	106,957	23.2	159,819

4 Security Analysis

4.1 Security Assumptions

In this section, we delineate the hard problems underpinning the security of Mito and analyze their complexity. The security of our cryptosystem fundamentally relies on the hardness of the Syndrome Decoding Problem (SDP) for linear codes endowed with a quasi-dyadic structure.

4.1.1 Cryptographic Definitions

Throughout, λ denotes the security parameter. We formally define the syntax and security requirements for **Public-Key Encryption** (PKE) and KEM.

Definition 4.1.1 (Public-Key Encryption). A *public-key encryption scheme* is a tuple of polynomial-time algorithms $\Pi = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt})$ defined as follows:

- $\text{KeyGen}(1^\lambda) \rightarrow (\text{pk}, \text{sk})$: A probabilistic algorithm that takes the security parameter as input and outputs a public key pk and a secret key sk .
- $\text{Encrypt}(\text{pk}, \mathbf{m}, \theta) \rightarrow \mathbf{c}$: A deterministic algorithm that takes a public key pk , a message $\mathbf{m} \in \mathcal{M}$, and randomness $\theta \in \mathcal{R}$ as input, and outputs a ciphertext $\mathbf{c}$.
- $\text{Decrypt}(\text{sk}, \mathbf{c}) \rightarrow \mathbf{m}$ or $\perp$: A deterministic algorithm that takes a secret key sk and a ciphertext $\mathbf{c}$ as input, and outputs a message $\mathbf{m}$ or a failure symbol $\perp$.

In particular, for any $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$, and any message $\mathbf{m} \in \mathcal{M}$, it holds that

$$\mathbb{P} \left[\theta \xleftarrow{\$} \mathcal{R} : \text{Decrypt}(\text{sk}, \text{Encrypt}(\text{pk}, \mathbf{m}, \theta)) = \mathbf{m} \right] \geq 1 - \text{negl}(\lambda),$$

where $\text{negl}(\lambda)$ is a negligible function of λ .

Definition 4.1.2 (IND-CPA Security for PKE). The IND-CPA security of a PKE scheme Π is defined by the following experiment $\text{Exp}_{\Pi, \mathcal{A}}^{\text{IND-CPA}}(\lambda)$:

$$\begin{array}{l} \text{Exp}_{\Pi, \mathcal{A}}^{\text{IND-CPA}}(\lambda) \\ \hline (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda) \\ (\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}(\text{pk}) \\ b \xleftarrow{\$} \mathbb{F}_2, \theta \xleftarrow{\$} \mathcal{R} \\ \mathbf{c}^* \leftarrow \text{Encrypt}(\text{pk}, \mathbf{m}_b, \theta) \\ b' \leftarrow \mathcal{A}(\mathbf{c}^*) \\ \text{return}(b' = b) \end{array}$$

The advantage of $\mathcal{A}$ is defined as $\text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-CPA}}(\lambda) := |\mathbb{P}[b' = b] - 1/2|$. And the scheme Π is called *IND-CPA secure* if this advantage is negligible.

In the following, we present a generalization of the quasi-cyclic scheme [AMBD⁺18].

Example 4.1.1 (Fully Regular Scheme). Let $m \in \mathbb{N}$, and let $\mathbb{F}_2^m$ be an m -dimensional linear space equipped with a norm $|\cdot|$. Let $\mathcal{S}(\mathbf{P})$ denote the subset of elements in $\mathbb{F}_2^m$ whose norms satisfy the constraint $\mathbf{P}$. Suppose $\xi : \mathbb{F}_2^m \times \mathbb{F}_2^m \rightarrow \mathbb{F}_2^m$ is a *symmetric bilinear map*, and there exists an identity element $\text{id} \in \mathbb{F}_2^m$ such that for all $y \in \mathbb{F}_2^m$, $\xi(\text{id}, y) = y$. Let $\mathcal{C}$ be an $[m, k]$ -linear code equipped with an encoding algorithm Encode and a decoding algorithm Decode , whose error-correcting capability is defined with respect to the norm $|\cdot|$. Let $A \subset \mathbb{F}_2^{m \times m}$ be an *admissible matrix space*.

Then, a *fully regular scheme* $\text{FRS}(\mathbf{P}_x, \mathbf{P}_y, \mathbf{P}_r, \mathbf{P}_t, \mathbf{P}_e, \xi, A, \mathcal{C})$ is a PKE scheme defined as follows.

FRS.KeyGen(1^λ)	FRS.Encrypt(pk, m, θ)	FRS.Decrypt(sk, c)
1. $H \xleftarrow{\\$} A$ 2. $\mathbf{x} \xleftarrow{\\$} \mathcal{S}(\mathbf{P}_x), \mathbf{y} \xleftarrow{\\$} \mathcal{S}(\mathbf{P}_y)$ 3. $\mathbf{s} \leftarrow \mathbf{x} + \mathbf{y}H$ 4. $\text{pk} \leftarrow (\mathbf{s}, H)$ 5. $\text{sk} \leftarrow \mathbf{y}$ 6. return (pk, sk)	1. $\mathbf{r} \xleftarrow{\theta} \mathcal{S}(\mathbf{P}_r), \mathbf{t} \xleftarrow{\theta} \mathcal{S}(\mathbf{P}_t)$ 2. $(\mathbf{s}, H) \leftarrow \text{pk}$ 3. $\mathbf{u} \leftarrow \mathbf{r} + \mathbf{t}H$ 4. $\mathbf{e} \xleftarrow{\theta} \mathcal{S}(\mathbf{P}_e)$ 5. $\mathbf{v} \leftarrow \text{Encode}(\mathbf{m}) + \mathbf{e} + \xi(\mathbf{t}, \mathbf{s})$ 6. $\mathbf{c} \leftarrow (\mathbf{u}, \mathbf{v})$ 7. return c	1. $\mathbf{y} \leftarrow \text{sk}$ 2. $(\mathbf{u}, \mathbf{v}) \leftarrow \mathbf{c}$ 3. $\mathbf{m} \leftarrow \text{Decode}(\mathbf{v} - \xi(\mathbf{y}, \mathbf{u}))$ 4. return m

Figure 4.1. The Fully Regular Encryption Scheme $\text{FRS}(\mathbf{P}_x, \mathbf{P}_y, \mathbf{P}_r, \mathbf{P}_t, \mathbf{P}_e, \xi, A, \mathcal{C})$

To ensure that the receiver can correctly decrypt the message $\mathbf{m}$, we require the weight of the *equivalent error*, defined as

$$\mathbf{e}_{\text{eq}} := \mathbf{e} + \xi(\mathbf{t}, \mathbf{s}) - \xi(\mathbf{y}, \mathbf{u}),$$

to be sufficiently small such that the decoding capability of the linear code $\mathcal{C}$ is not exceeded. To achieve this, we require ξ to satisfy the following property:

$$\xi(\mathbf{x}, \mathbf{y}H) = \xi(\mathbf{y}, \mathbf{x}H), \quad \forall \mathbf{x}, \mathbf{y} \in \mathbb{F}_2^m, H \in A. \quad (4.1)$$

With this property, the equivalent error cleanly simplifies to

$$\begin{aligned} \mathbf{e}_{\text{eq}} &= \mathbf{e} + \xi(\mathbf{t}, \mathbf{x} + \mathbf{y}H) - \xi(\mathbf{y}, \mathbf{r} + \mathbf{t}H) \\ &= \mathbf{e} + \xi(\mathbf{t}, \mathbf{x}) - \xi(\mathbf{y}, \mathbf{r}) + \xi(\mathbf{t}, \mathbf{y}H) - \xi(\mathbf{y}, \mathbf{t}H) \\ &= \mathbf{e} + \xi(\mathbf{t}, \mathbf{x}) - \xi(\mathbf{y}, \mathbf{r}). \end{aligned}$$

Specifically, since ξ is a symmetric bilinear map, there exists a linear mapping $\Xi : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^{m \times m}$ such that $\xi(\mathbf{x}, \mathbf{y}) = \mathbf{x} \cdot \Xi(\mathbf{y})$. Furthermore, Equation (4.1) implies that

$$H \cdot \Xi(\mathbf{y}) = \Xi(\mathbf{y}H), \quad \forall \mathbf{y} \in \mathbb{F}_2^m.$$

By setting $\mathbf{y} = \text{id}$, we have

$$H = H \cdot \Xi(\text{id}) = \Xi(\text{id} \cdot H).$$

This demonstrates that any matrix $H \in A$ must be the image of some vector $\mathbf{h} \in \mathbb{F}_2^m$ under the mapping Ξ , i.e., $H = \Xi(\mathbf{h})$. Hence, we assume that $A = \text{Im } \Xi$. Consequently, we can rewrite (4.1) as

$$\xi(\mathbf{x}, \xi(\mathbf{y}, \mathbf{h})) = \xi(\mathbf{y}, \xi(\mathbf{x}, \mathbf{h})), \quad \forall \mathbf{x}, \mathbf{y}, \mathbf{h} \in \mathbb{F}_2^m.$$

We recover the standard *associativity* by

$$\xi(\xi(\mathbf{x}, \mathbf{y}), \mathbf{h}) = \xi(\mathbf{h}, \xi(\mathbf{x}, \mathbf{y})) = \xi(\mathbf{x}, \xi(\mathbf{h}, \mathbf{y})) = \xi(\mathbf{x}, \xi(\mathbf{y}, \mathbf{h})).$$

This reveals that $(\mathbb{F}_2^m, +, \xi)$ forms a commutative unital ring. By the category equivalence between finite $\mathbb{F}_2$ -algebras and polynomial quotient rings over $\mathbb{F}_2$ with finite dimension, the bilinear map ξ is intrinsically determined by the generator ideal I of some polynomial quotient ring $\mathcal{R} = \mathbb{F}_2[x_0, x_1, \dots, x_\ell]/I$ satisfying $I \cap \mathbb{F}_2[x_i] \neq \{0\}$ for all $i \in [\ell]$. In this context, the matrix $\Xi(\mathbf{y})$ is exactly the *regular representation* of $\mathbf{y}$, i.e., the matrix representation of the left-multiplication operator $L_{\mathbf{y}}$ of the element $\mathbf{y} \bmod I$. This algebraic correspondence is why we refer to this construction as a “regular” scheme.

In particular, a **Fully Regular Scheme** (FRS) with $\ell = 0$ and $I = \langle x_0^n + 1 \rangle$ is called a *quasi-cyclic scheme*, and an FRS induced by dyadic algebra $\mathcal{R}_\ell$ is called a *quasi-dyadic scheme*. $\triangle$

In practice, the PKE procedure in [GAMA⁺25] differs slightly from our aforementioned quasi-cyclic framework. During the encryption phase, the transmitted ciphertext (specifically, the component $\mathbf{v}$) is actually *truncated*. The rationale for this difference stems from security considerations: the dimension n must be chosen as a prime for which 2 is a primitive root, whereas the length n' of the concatenated code $\mathcal{C}$ used for error correction is inherently a composite number. Consequently, the vector $\mathbf{e} + \xi(\mathbf{t}, \mathbf{s})$ is effectively truncated by $(n - n')$ bits.

More generally, we formalize this via the following definition:

Definition 4.1.3 (Projectively Regular Scheme). Let $\mathbb{F}_2^m$ be a linear space equipped with a norm $|\cdot|$. Let $\mathcal{S}(\mathbf{P})$ denote the subset of elements whose norms satisfy the constraint $\mathbf{P}$. Suppose $\xi : \mathbb{F}_2^m \times \mathbb{F}_2^m \rightarrow \mathbb{F}_2^m$ is a symmetric bilinear map induced by an isomorphism between $\mathbb{F}_2$ -algebras

$$\mathbb{F}_2^m \simeq \mathbb{F}_2[x_0, x_1, \dots, x_\ell]/I.$$

Let $\pi : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^n$ be a projection map for $n \leq m$ and $\mathcal{C}$ be an $[n', k]$ -linear code with an encoding algorithm Encode and a decoding algorithm Decode, whose error-correcting capability is defined with respect to the norm $|\cdot|$. Then, a *projectively regular scheme* $\text{PRS}(\mathbf{P}_x, \mathbf{P}_y, \mathbf{P}_r, \mathbf{P}_t, \mathbf{P}_e, \xi, \pi, \mathcal{C})$ is a PKE scheme defined as follows.

PRS.KeyGen(1^λ)	PRS.Encrypt(pk, $\mathbf{m}$, θ)	PRS.Decrypt(sk, $\mathbf{c}$)
1. $H \xleftarrow{\\$} \mathbf{A}$ 2. $\mathbf{x} \xleftarrow{\\$} \mathcal{S}(\mathbf{P}_x), \mathbf{y} \xleftarrow{\\$} \mathcal{S}(\mathbf{P}_y)$ 3. $\mathbf{s} \leftarrow \mathbf{x} + \mathbf{y}H$ 4. $\text{pk} \leftarrow (\mathbf{s}, H)$ 5. $\text{sk} \leftarrow \mathbf{y}$ 6. return (pk, sk)	1. $\mathbf{r} \xleftarrow{\theta} \mathcal{S}(\mathbf{P}_r), \mathbf{t} \xleftarrow{\theta} \mathcal{S}(\mathbf{P}_t)$ 2. $(\mathbf{s}, H) \leftarrow \text{pk}$ 3. $\mathbf{u} \leftarrow \mathbf{r} + \mathbf{t}H$ 4. $\mathbf{e} \xleftarrow{\theta} \mathcal{S}(\mathbf{P}_e)$ 5. $\mathbf{v} \leftarrow \text{Encode}(\mathbf{m}) + \pi(\mathbf{e} + \xi(\mathbf{t}, \mathbf{s}))$ 6. $\mathbf{c} \leftarrow (\mathbf{u}, \mathbf{v})$ 7. return $\mathbf{c}$	1. $\mathbf{y} \leftarrow \text{sk}$ 2. $(\mathbf{u}, \mathbf{v}) \leftarrow \mathbf{c}$ 3. $\mathbf{m} \leftarrow \text{Decode}(\mathbf{v} - \pi(\xi(\mathbf{y}, \mathbf{u})))$ 4. return $\mathbf{m}$

Figure 4.2. The Projectively Regular Scheme (PRS)

Here, the equivalent error is defined as

$$\mathbf{e}_{\text{eq}} := \pi(\mathbf{e}) + \pi(\xi(\mathbf{t}, \mathbf{s})) - \pi(\xi(\mathbf{y}, \mathbf{u})).$$

From (4.1), we observe the symmetry $\pi(\xi(\mathbf{t}, \mathbf{y}H)) = \pi(\xi(\mathbf{y}, \mathbf{t}H))$, which allows us to reduce the expression to

$$\mathbf{e}_{\text{eq}} = \pi(\mathbf{e}) + \pi(\xi(\mathbf{t}, \mathbf{x})) - \pi(\xi(\mathbf{y}, \mathbf{r})).$$

As demonstrated in [GAMA⁺25, Section 6.2.1], the IND-CPA security of a PRS follows from that of its corresponding FRS. This holds because any instance of a PRS can be viewed as an instance of the FRS with a portion of its information bits erased. The projective structure reduces the *communication overhead* of the scheme. By choosing a projection π with a sufficiently small image space, the ciphertext size can be substantially reduced from $2m$ to $m + n'$. For more details, see Section 4.1.4.

Remark 4.1.1. From the perspective of coding theory, the matrix $H \in \mathbf{A}$ utilized in the FRS (resp., PRS) is *reproducible*. This holds because, given a basis $\{\text{id}, e_1, \dots, e_{m-1}\}$ of the quotient ring $\mathcal{R}$, L_h serves as the regular representation of h . The i -th row of L_h corresponds to the product $\xi(e_i, h)$. Hence, this row can be expressed as the first row v_h acted upon by the linear transformation L_{e_i} , namely $v_h \cdot L_{e_i}$.

Definition 4.1.4 (Key Encapsulation Mechanism). A *key encapsulation mechanism* is a tuple of polynomial-time algorithms $\Gamma = (\text{KeyGen}, \text{Encaps}, \text{Decaps})$ defined as follows:

- $\text{KeyGen}(1^\lambda) \rightarrow (\text{pk}, \text{sk})$: A probabilistic algorithm that outputs a public key pk and a secret key sk .
- $\text{Encaps}(\text{pk}) \rightarrow (K, \text{ct})$: A probabilistic algorithm that takes a public key pk as input, and outputs a shared secret key $K \in \mathcal{K}$ and an encapsulation ciphertext ct .
- $\text{Decaps}(\text{sk}, \text{ct}) \rightarrow K$ or $\perp$: A deterministic algorithm that takes a secret key sk and an encapsulation ct as input, and returns a shared key K or a failure symbol $\perp$.

In particular, it holds that

$$\mathbb{P}[(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda); (K, \text{ct}) \leftarrow \text{Encaps}(\text{pk}) : \text{Decaps}(\text{sk}, \text{ct}) = K] \geq 1 - \text{negl}(\lambda).$$

Definition 4.1.5 (IND-CCA Security for KEM). The IND-CCA security of a KEM scheme Γ is defined by the following experiment $\text{Exp}_{\Gamma, \mathcal{A}}^{\text{IND-CCA}}(\lambda)$:

$$\begin{array}{l} \text{Exp}_{\Gamma, \mathcal{A}}^{\text{IND-CCA}}(\lambda) \\ \hline (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda) \\ (K_0, \text{c}^*) \leftarrow \text{Encaps}(\text{pk}) \\ K_1 \xleftarrow{\$} \mathcal{K}, b \xleftarrow{\$} \mathbb{F}_2 \\ b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{decaps}}}(\text{pk}, \text{c}^*, K_b) \\ \text{return}(b' = b) \end{array}$$

The decapsulation oracle $\mathcal{O}_{\text{decaps}}(\text{c})$ returns $\text{Decaps}(\text{sk}, \text{c})$ for any $\text{c} \neq \text{c}^*$, and $\perp$ if $\text{c} = \text{c}^*$. The advantage of $\mathcal{A}$ is defined as

$$\text{Adv}_{\Gamma, \mathcal{A}}^{\text{IND-CCA}}(\lambda) = \left| \mathbb{P}[b' = b] - \frac{1}{2} \right|.$$

The scheme Γ is *IND-CCA secure* if this advantage is negligible for all bounded adversaries.

Definition 4.1.6 (Salted FO Transform with Implicit Rejection, [GHS25, Definition 10]). Let Π be a PKE scheme, λ_{salt} be a salt length parameter, and $\mathcal{G}_{\text{FO}}, \mathcal{H}_{\text{FO}}$ be hash functions. The salted FO transform yields $\text{KEM}^\perp := \text{SFO}^\perp[\Pi, \mathcal{G}_{\text{FO}}, \mathcal{H}_{\text{FO}}, \lambda_{\text{salt}}]$ defined as follows.

$\text{KEM}^\perp.\text{KeyGen}(1^\lambda)$	$\text{KEM}^\perp.\text{Encaps}(\text{pk})$	$\text{KEM}^\perp.\text{Decaps}(\text{pk}, \text{sk}', \text{ct})$
1. $(\text{pk}, \text{sk}) \leftarrow \Pi.\text{KeyGen}(1^\lambda)$ 2. $\text{s} \xleftarrow{\\$} \mathcal{M}$ 3. $\text{sk}' \leftarrow (\text{sk}, \text{s})$ 4. return (pk, sk')	1. $\text{m} \xleftarrow{\\$} \mathcal{M}, \text{salt} \xleftarrow{\\$} \mathbb{F}_2^{\lambda_{\text{salt}}}$ 2. $\theta \leftarrow \mathcal{G}_{\text{FO}}(\text{pk}, \text{m} \parallel \text{salt})$ 3. $\text{c} \leftarrow \Pi.\text{Encrypt}(\text{pk}, \text{m}, \theta)$ 4. $K \leftarrow \mathcal{H}_{\text{FO}}(\text{pk}, \text{m}, \text{c} \parallel \text{salt})$ 5. $\text{ct} \leftarrow (\text{c}, \text{salt})$ 6. return (K, ct)	1. $(\text{sk}, \text{s}) \leftarrow \text{sk}', (\text{c}, \text{salt}) \leftarrow \text{ct}$ 2. $\text{m}' \leftarrow \Pi.\text{Decrypt}(\text{sk}, \text{c})$ 3. $\theta' \leftarrow \mathcal{G}_{\text{FO}}(\text{pk}, \text{m}' \parallel \text{salt})$ 4. $\text{c}' \leftarrow \Pi.\text{Encrypt}(\text{pk}, \text{m}', \theta')$ 5. if $\text{m}' = \perp$ or $\text{c} \neq \text{c}'$ then 6. return $\mathcal{H}_{\text{FO}}(\text{pk}, \text{s}, \text{c} \parallel \text{salt})$ 7. return $\mathcal{H}_{\text{FO}}(\text{pk}, \text{m}', \text{c} \parallel \text{salt})$

Figure 4.3. The Implicit Rejection Fujisaki-Okamoto Transform ($\text{KEM}^\perp$)

4.1.2 Syndrome Decoding Problem

The security of code-based cryptosystems relies fundamentally on the hardness of decoding random linear codes, a problem historically known as the **Syndrome Decoding Problem** (SDP). For $n \in \mathbb{N}$, we denote by $\mathcal{S}(w)$ the set of vectors in $\mathbb{F}_2^n$ with Hamming weight exactly w .

Definition 4.1.7 (SD Distribution). Let n, k, w be positive integers with $k < n$ and $w < n$. The **Syndrome Decoding Distribution**, denoted $\text{SD}(n, k, w)$, generates a problem instance by sampling a parity-check matrix $H \xleftarrow{\$} \mathbb{F}_2^{(n-k) \times n}$ uniformly at random and an error vector $\mathbf{e} \xleftarrow{\$} \mathcal{S}(w)$, then computing the syndrome $\mathbf{s}^\top = H\mathbf{e}^\top$, and finally outputting the pair $(H, \mathbf{s})$.

Based on this distribution, we define the decisional version of the problem, which serves as the basis for the semantic security of the cryptosystem.

Definition 4.1.8 (Decisional SD Problem). Let n, k, w be positive integers. Given a tuple $(H, \mathbf{s}) \in \mathbb{F}_2^{(n-k) \times n} \times \mathbb{F}_2^{n-k}$, the **Decisional Syndrome Decoding Problem** (D-SDP) $\text{D-SDP}(n, k, w)$ asks to decide with non-negligible advantage whether $(H, \mathbf{s})$ was sampled from the distribution $\text{SD}(n, k, w)$ or drawn from the uniform distribution over $\mathbb{F}_2^{(n-k) \times n} \times \mathbb{F}_2^{n-k}$.

It is a classical result that the search version of SDP (S-SDP), i.e., finding $\mathbf{e}$ for given $H, \mathbf{s}$, is NP-complete in the worst case. Furthermore, it has been established that the D-SDP is computationally equivalent to the S-SDP via a polynomial-time reduction. The most efficient algorithms for solving the SDP belong to the class of **Information Set Decoding** (ISD). Since the pioneering work of Prange [Pra62], numerous improvements have been proposed, including the algorithms by Stern [Ste88], Dumer [Dum86], and more recently, the BJMM [BJMM12], MMT [MMT11] and nearest neighbours search [MO15, BM17] variants. Despite these optimizations, the time complexity of the best-known ISD algorithms remains exponential in the weight w , typically following the form $2^{c \cdot w(1+o(1))}$ for some constant c determined by the code rate k/n . This holds up to a polynomial factor of $O\left(\frac{1}{\sqrt{n}}\right)$ as introduced by the **Decoding One Out of Many** (DOOM) framework [Sen11]. We therefore use the work factor of the most efficient generic ISD algorithm as the benchmark for evaluating the concrete security level of our proposed scheme.

4.1.3 HQC Assumption

Before formally presenting the SDP for Mito, we briefly review the security foundations of the quasi-cyclic scheme [AMBD⁺18]. As mentioned in **Example 4.1.1**, quasi-cyclic schemes can be viewed as a special case of FRS, specifically under the $\mathbb{F}_2$ -algebra isomorphism $\mathbb{F}_2^n \simeq \mathcal{R}_0$. Specifically, HQC instantiates a quasi-cyclic scheme by setting the norms to Hamming weights $w(\mathbf{x}) = w(\mathbf{y}) = w$ and $w(\mathbf{r}) = w(\mathbf{t}) = w(\mathbf{e}) = w_r$.

The security of HQC [GAMA⁺25] reduces to the hardness of two variants of the **Decisional Quasi-Cyclic Syndrome Decoding** (DQCSD) problem:

- the DQCSD with Parity (DQCSD-P) problem $s\text{-DQCSD-P}(\omega, \mathbf{b})$ determined by QC dimension $s \geq 2$, given weight $\omega \geq 1$ and parity vector $\mathbf{b} = (b_1, b_2, \dots, b_{s-1}) \in \mathbb{F}_2^{s-1}$;
- the DQCSD with Parity and Truncation (DQCSD-PT) problem $s\text{-DQCSD-PT}(\omega, \mathbf{b})$ determined by QC dimension $s \geq 2$, given weight $\omega \geq 1$, parity vector $\mathbf{b} = (b_1, b_2, \dots, b_{s-1}) \in \mathbb{F}_2^{s-1}$ and projective map $\pi : \mathcal{R}_0^{s-1} \rightarrow \mathbb{F}_2^{n'}$.

For $b \in \mathbb{F}_2$, denote the subset of ring elements with parity b as

$$\mathcal{R}_{0,b} = \{r(x) \in \mathcal{R}_0 \mid r(1) = b\}.$$

We formally define the underlying distributions of the above problems.

Definition 4.1.9 ($s\text{-DQCSD-P}$ and $s\text{-DQCSD-PT}$ Distributions). Let $\omega, s, \mathbf{b}$, and π be defined as above. The distributions $s\text{-DQCSD-P}(\omega, \mathbf{b})$ and $s\text{-DQCSD-PT}(\omega, \mathbf{b}, \pi)$ generate an instance $(H, \mathbf{s})$ as follows:

$s\text{-QCSD-P}(\omega, \mathbf{b})$	$s\text{-QCSD-PT}(\omega, \mathbf{b}, \pi)$
for $i = 1, \dots, s-1$:	for $i = 1, \dots, s-1$:
$h_i \xleftarrow{\$} \mathcal{R}_{0, b_i}$	$h_i \xleftarrow{\$} \mathcal{R}_{0, b_i}$
$\mathbf{h} \leftarrow (h_1, \dots, h_{s-1})$	$\mathbf{h} \leftarrow (h_1, \dots, h_{s-1})$
$H \leftarrow [I_{s-1} \mid \mathbf{h}^\top]$	$H \leftarrow [I_{s-1} \mid \mathbf{h}^\top]$
for $j = 1, \dots, s$:	for $j = 1, \dots, s$:
$e_j \xleftarrow{\$} \mathcal{R}_{0, \omega}$	$e_j \xleftarrow{\$} \mathcal{R}_{0, \omega}$
$\mathbf{e} \leftarrow (e_1, \dots, e_s)$	$\mathbf{e} \leftarrow (e_1, \dots, e_s)$
$\mathbf{s} \leftarrow \mathbf{e}H^\top$	$\mathbf{s} \leftarrow \pi(\mathbf{e}H^\top)$
return $(H, \mathbf{e})$	return $(H, \mathbf{e})$

For a positive integer ω and a parity vector $\mathbf{b} \in \mathbb{F}_2^{s-1}$, the **derived syndrome parity** vector is defined as $\mathbf{p}(\omega, \mathbf{b}) := (b'_1, \dots, b'_{s-1})$, where $b'_i = \omega + b_i \cdot \omega \pmod{2}$.

For $\dagger \in \{s\text{-DQCSD-P}, s\text{-DQCSD-PT}\}$, let $\mathcal{D}_\dagger$ denote the distributions $s\text{-QCSD-P}(\omega, \mathbf{b})$ and $s\text{-QCSD-PT}(\omega, \mathbf{b}, \pi)$, and $\mathcal{D}_{\text{rand}}$ denote the uniform distribution over the space of valid systematic matrices $H = [I_{s-1} \mid \mathbf{h}^\top]$ and random vectors $\mathbf{s} \in \mathcal{R}_{0, \mathbf{b}}^{s-1} := \prod \mathcal{R}_{0, b'_i}$ (resp. $\pi(\mathcal{R}_{0, \mathbf{b}}^{s-1})$). For any adversary $\mathcal{A}$, the distinguishing advantage for the problem $\dagger \in \{s\text{-DQCSD-P}, s\text{-DQCSD-PT}\}$ is defined as

$$\text{Adv}_{\dagger, \mathcal{A}} := \left| \mathbb{P}[(H, \mathbf{s}) \leftarrow \mathcal{D}_\dagger : \mathcal{A}(H, \mathbf{s}) = 1] - \mathbb{P}[(H, \mathbf{s}) \leftarrow \mathcal{D}_{\text{rand}} : \mathcal{A}(H, \mathbf{s}) = 1] \right|.$$

For $\omega = \omega(\lambda)$, the $s\text{-DQCSD-P}(\omega, \mathbf{b})$ (resp. $s\text{-DQCSD-PT}(\omega, \mathbf{b}, \pi)$) problem asks whether there exists a bounded adversary $\mathcal{A}$ with non-negligible advantage in the security parameter λ .

The overall security of the HQC is established from the following theorem:

Theorem 4.1.1 (IND-CPA Security of HQC.PKE, [GAMA⁺25, Theorem 6.2]). For any IND-CPA adversary $\mathcal{A}$ against the HQC.PKE, there exist an adversary $\mathcal{B}_1$ against the 2-DQCSD-P($w, \mathbf{b}$) problem and an adversary $\mathcal{B}_2$ against the 3-DQCSD-PT($w_r, \mathbf{b}, \pi$) problem, such that

$$\text{Adv}_{\text{HQC}, \mathcal{A}}^{\text{IND-CPA}}(\lambda) \leq \text{Adv}_{2\text{-DQCSD-P}, \mathcal{B}_1}(\lambda) + \text{Adv}_{3\text{-DQCSD-PT}, \mathcal{B}_2}(\lambda).$$

Remark 4.1.2. Since the parameter choices in HQC assign identical Hamming weights to the respective secret vector components in both QCSD problems, the scheme is said to have **balanced parameters**.

To bound the adversary's advantage for a given security level, HQC treats the QCSD problem as an SDP over quasi-cyclic codes. Because introducing an algebraic structure means the problem is no longer guaranteed to be NP-complete, it necessitates the following assumption:

Assumption 4.1.1 (HQC Assumption, [AMBD⁺18, Assumption 1]). Let n, s, t, w be positive integers. Denote $H \in (\mathbb{F}_2^n)^{t \times s}$ to be a matrix chosen uniformly at random from the set of systematic quasi-cyclic matrices. The problem of finding a low-weight composite vector $\mathbf{er} = (\mathbf{x}, \mathbf{y}) \in (\mathbb{F}_2^n)^s$, where $\mathbf{x} \in (\mathbb{F}_2^n)^{s-t}$ and $\mathbf{y} \in (\mathbb{F}_2^n)^t$ with total Hamming weight $w(\mathbf{er}) = w$, satisfying the syndrome equation

$$\mathbf{er} \cdot H^\top = \mathbf{s}$$

is computationally equivalent to solving the generic syndrome decoding problem $\text{SDP}(sn, tn, w)$ over $\mathbb{F}_2$ up to a small (polynomial) factor.

In other words, the quasi-cyclic structure yields no algorithmic advantage significantly superior to generic ISD algorithms.

To ensure the validity of **Assumption 4.1.1**, HQC introduces two structural trade-offs:

Consistency with the public part: HQC must rely on the SDP *with parity* as the underlying hard problem for its reduction. Indeed, the specific subset of ring elements with parity $\mathcal{R}_{0,b}$ to which each block of the syndrome $\mathbf{s}$ belongs is strictly determined once the public vector $\mathbf{h}$ and the weight constraint on the error vector $\mathbf{e}$ are fixed.

Indecomposability of the private part: When constructing the $\mathbb{F}_2$ -algebra isomorphism $\mathbb{F}_2^n \simeq \mathcal{R}_0$, the $\mathbb{F}_2$ -dimension n is restricted to an odd prime for which 2 is a primitive root. This restriction is necessary because attacks targeting quasi-cyclic structures become highly efficient when the polynomial $x^n - 1$ factors into many low-degree components [GJL15, LJKS⁺16].

Via the Chinese Remainder Theorem (CRT), the two conditions discussed above map to the respective components of the ring decomposition

$$\mathcal{R}_0 \simeq \mathcal{R}_0 / \langle X + 1 \rangle \times \mathcal{R}_0 / \langle \Phi_n(X) \rangle \simeq \mathbb{F}_2 \times \mathbb{F}_{2^{n-1}}.$$

More generally, scheme $\text{FRS}(\mathbf{P}_x, \mathbf{P}_y, \mathbf{P}_r, \mathbf{P}_t, \mathbf{P}_e, \xi, \mathcal{C})$ is said to be *provable* if its induced polynomial quotient ring $\mathcal{R}$ admits a decomposition of the form

$$\mathcal{R}_{\text{public}} \times \mathcal{R}_{\text{private}},$$

such that the constraint $\mathbf{P}_x, \mathbf{P}_y, \mathbf{P}_r, \mathbf{P}_t, \mathbf{P}_e$, on

$$\mathbf{s} = (\mathbf{x}, \mathbf{y}) \begin{pmatrix} 1 & \mathbf{h} \end{pmatrix}^\top \text{ and } \mathbf{u} = (\mathbf{r}, \mathbf{e}, \mathbf{t}) \begin{pmatrix} 1 & \mathbf{0} & \mathbf{h} \\ \mathbf{0} & 1 & \mathbf{s} \end{pmatrix}^\top$$

depends exclusively on the projection of $\mathbf{s}$ and $\mathbf{u}$ onto $\mathcal{R}_{\text{public}}$, and $\mathcal{R}_{\text{private}}$ is an *indecomposable* ring.

To conclude this subsection, we present the *main theorem* that underpins the security of Mito.

Theorem 4.1.2. The quasi-dyadic scheme $\text{FRS}(\mathcal{R}_\ell)$ induced by the dyadic algebra $\mathcal{R}_\ell$ is provable.

Proof. To show that FRS is a provable scheme, we must demonstrate that its induced ring admits the decomposition $\mathcal{R}_\ell \simeq \mathcal{R}_{\text{public}} \times \mathcal{R}_{\text{private}}$, where the weight constraint depends only on $\mathcal{R}_{\text{public}}$, and $\mathcal{R}_{\text{private}}$ is indecomposable.

First, by the Chinese remainder theorem, the dyadic algebra $\mathcal{R}_\ell$ decomposes into

$$\mathcal{R}_\ell \simeq \mathcal{R}_0 \otimes_{\mathbb{F}_2} \mathbb{D}_\ell \simeq \underbrace{\mathbb{D}_\ell}_{\mathcal{R}_{\text{public}}} \times \underbrace{(\mathbb{D}_\ell \otimes_{\mathbb{F}_2} \mathbb{F}_{2^{n-1}})}_{\mathcal{R}_{\text{private}}}, \quad (4.2)$$

where $\mathbb{D}_\ell = \mathbb{F}_2[T_1, \dots, T_\ell] / \langle T_1^2 + 1, \dots, T_\ell^2 + 1 \rangle$.

Viewing $\mathcal{R}_\ell$ as an $\mathcal{R}_0$ -module, the corresponding weight vector $\mathbf{w}(\mathbf{r})$ for any $\mathbf{r} \in \mathcal{R}_\ell$ is precisely determined by the evaluation map $\text{Eval}_{X=1}$. As discussed previously, the weight constraint on the error vectors acts exclusively through their projection onto $\mathbb{D}_\ell$.

Next, we establish the indecomposability of the private part via the following lemma.

Lemma 4.1.1. The ring $A := \mathbb{D}_\ell \otimes_{\mathbb{F}_2} \mathbb{F}_{2^{n-1}}$ is an indecomposable ring. That is, if there exist rings B and C such that $A \simeq B \times C$, then either $A \simeq B$ or $A \simeq C$.

Proof of Lemma 4.1.1. Recall that a commutative ring is decomposable if and only if it contains a non-trivial idempotent element e such that $e^2 = e$ and $e \notin \{0, 1\}$. Let $J = \langle T_1 + 1, \dots, T_\ell + 1 \rangle \subset A$ be the ideal generated by the nilpotent variables. The quotient ring $A/J \simeq \mathbb{F}_{2^{n-1}}$ is a field, in which the only idempotents are 0 and 1.

Since J is a nilpotent ideal, idempotents in A lift uniquely from idempotents in A/J . Specifically, if $e \in A$ is an idempotent, its image $\bar{e} = e + J \in A/J$ must be exactly 0 or 1. If $\bar{e} = 0$, then $e \in J$, implying that e is nilpotent. The only element that is simultaneously nilpotent and idempotent is 0, which forces $e = 0$. Similarly, if $\bar{e} = 1$, then $1 - e$ is a nilpotent idempotent, forcing $1 - e = 0$, hence $e = 1$. Thus, A contains only trivial idempotents, meaning it is indecomposable. $\triangle$

With Lemma 4.1.1 established, we conclude that the induced ring decomposes into a public component $\mathcal{R}_{\text{public}} = \mathbb{D}_\ell$ and an indecomposable private component $\mathcal{R}_{\text{private}} = A$. In particular, since $\mathcal{R}_\ell$ is a commutative Artinian ring, the structure theorem guarantees that its decomposition into indecomposable components is unique and corresponds to the localizations at its maximal ideals. Therefore, $\text{FRS}(\mathcal{R}_\ell)$ satisfies all the definitional requirements of a provable scheme. ■

4.1.4 Unbalanced Parameters and Projective Structure

In this section, we present the security reduction and complexity analysis of Mito, modeled as a projective regular scheme $\text{Mito}_\ell.\text{PKE} := \text{PRS}(\mathbf{P}_x, \mathbf{P}_y, \mathbf{P}_r, \mathbf{P}_t, \xi_\ell, \pi)$. Hereafter, for $\dagger \in \{x, y, r, t\}$, we define the constraint sets as

$$\mathcal{S}(\mathbf{P}_\dagger) = \mathcal{S}^D(\mathbf{w}_\dagger),$$

where $\mathbf{w}_\dagger \in \mathbb{N}^{2^\ell}$ denote the respective weight vectors. Let ξ_ℓ denote the bilinear map induced by $\mathcal{R}_\ell$, and $\text{Pr} : \mathcal{R}_\ell \rightarrow \mathcal{R}_0$ be the valuation map generated by $T_1 = 1$ and $T_i = 0$ for all $i \geq 2$. By viewing $\mathcal{R}_\ell$ as an $\mathcal{R}_0$ -module, Pr is explicitly given by the mapping

$$(r_0, r_1, \dots, r_{2^\ell-1}) \mapsto r_0.$$

Furthermore, for any given parity vector $\mathbf{b} \in (\mathbb{F}_2)^{2^\ell}$, we define the parity-constrained subset of ring elements as

$$\mathcal{R}_\ell[\mathbf{b}] = \{u \in \mathcal{R}_\ell \mid \text{Eval}_{X=1}(u) = \mathbf{b}\}.$$

To account for the effect of weight constraints on the public components, we formalize the corresponding SD problems as follows.

Definition 4.1.10 (((s, ℓ) -QDSD-P and (s, ℓ) -QDSD-PT Distributions). Let ℓ and s be positive integers with $s \geq 2$. Let

$$\mathbf{w} = (\mathbf{w}_1, \dots, \mathbf{w}_s) \in (\mathbb{N}^{2^\ell})^s \quad \text{and} \quad \mathbf{b} = (\mathbf{b}_1, \dots, \mathbf{b}_{s-1}) \in (\mathbb{F}_2^{2^\ell})^{s-1}$$

be a weight tuple and a parity tuple, respectively. Let π be the projective mapping defined as $\pi = \text{Trun} \circ \text{Pr}$, where Trun denotes the function that truncates the rightmost $(n - n')$ bits of an n -bit string. The distribution (s, ℓ) -QDSD-P($\mathbf{w}, \mathbf{b}$) and the distribution (s, ℓ) -QDSD-PT($\mathbf{w}, \mathbf{b}, \pi$) generate problem instances $(H, \mathbf{s})$ according to the following procedures:

(s, ℓ) -QDSD-P($n, \mathbf{w}, \mathbf{b}$)

for $i = 1, \dots, s-1$:

$$h_i \xleftarrow{\$} \mathcal{R}_\ell[\mathbf{b}_i]$$

$$\mathbf{h} \leftarrow (h_1, \dots, h_{s-1})$$

$$H \leftarrow [I_{s-1} \mid \mathbf{h}^\top]$$

for $j = 1, \dots, s$:

$$e_j \xleftarrow{\$} \mathcal{R}_\ell[\mathbf{w}_j]$$

$$\mathbf{e} \leftarrow (e_1, \dots, e_s)$$

$$\mathbf{s} \leftarrow \mathbf{e}H^\top$$

return $(H, \mathbf{s})$

(s, ℓ) -QDSD-PT($n, \mathbf{w}, \mathbf{b}, \pi$)

for $i = 1, \dots, s-1$:

$$h_i \xleftarrow{\$} \mathcal{R}_\ell[\mathbf{b}_i]$$

$$\mathbf{h} \leftarrow (h_1, \dots, h_{s-1})$$

$$H \leftarrow [I_{s-1} \mid \mathbf{h}^\top]$$

for $j = 1, \dots, s$:

$$e_j \xleftarrow{\$} \mathcal{R}_\ell[\mathbf{w}_j]$$

$$\mathbf{e} \leftarrow (e_1, \dots, e_s)$$

$$\mathbf{s} \leftarrow \pi(\mathbf{e}H^\top)$$

return $(H, \mathbf{s})$

Given the weight tuple $\mathbf{w}$ and the parity tuple $\mathbf{b}$, the *derived syndrome parity* tuple $\mathbf{b}' \in (\mathbb{F}_2^{2^\ell})^{s-1}$ is defined as $\mathbf{b}' := (\mathbf{b}'_1, \dots, \mathbf{b}'_{s-1})$, where

$$\mathbf{b}'_i \equiv \mathbf{w}_i + \mathbf{b}_i \cdot \mathbf{w}_s \pmod{(2\mathbb{Z})^{2^\ell}}$$

for each $i \in \{1, \dots, s-1\}$.

For the problem variant $\dagger \in \{(s, \ell)\text{-DQDSD-P}, (s, \ell)\text{-DQDSD-PT}\}$, let $\mathcal{D}_\dagger$ denote the real distribution $(s, \ell)\text{-QDSD-P}(\mathbf{w}, \mathbf{b})$ or $(s, \ell)\text{-QDSD-PT}(\mathbf{w}, \mathbf{b}, \pi)$, respectively. Correspondingly, let $\mathcal{D}_{\text{rand}}$ denote the uniform distribution over the space of valid instances $(H, \mathbf{s})$, where $H = [I_{s-1} \mid \mathbf{h}^\top]$ is a valid systematic matrix and $\mathbf{s}$ is sampled uniformly from $\prod \mathcal{R}_\ell[\mathbf{b}'_i]$ (resp. $\pi(\prod \mathcal{R}_\ell[\mathbf{b}'_i])$). For any adversary $\mathcal{A}$, the distinguishing advantage against the problem $\dagger$ is formulated as:

$$\text{Adv}_{\dagger, \mathcal{A}} := \left| \mathbb{P}[(H, \mathbf{s}) \leftarrow \mathcal{D}_\dagger : \mathcal{A}(H, \mathbf{s}) = 1] - \mathbb{P}[(H, \mathbf{s}) \leftarrow \mathcal{D}_{\text{rand}} : \mathcal{A}(H, \mathbf{s}) = 1] \right|.$$

For parameterized weights $\mathbf{w} = \mathbf{w}(\lambda)$, the $(s, \ell)\text{-DQDSD-P}(\mathbf{w}, \mathbf{b})$ (resp. $(s, \ell)\text{-DQDSD-PT}(\mathbf{w}, \mathbf{b}, \pi)$) problem asks whether there exists a computationally bounded adversary $\mathcal{A}$ that achieves a non-negligible advantage in the security parameter λ .

Given that the specific choice of regular representation does not impact the security reduction, the following corollary is a straightforward consequence of [Theorem 4.1.1](#).

Corollary 4.1.1. For any IND-CPA adversary $\mathcal{A}$ against Mito.PKE, there exist an adversary $\mathcal{B}_1$ against the $(2, \ell)\text{-DQDSD-P}(\mathbf{w}_2, \mathbf{b})$ problem for $\mathbf{w}_2 := (\mathbf{w}_x, \mathbf{w}_y)$ and an adversary $\mathcal{B}_2$ against the $(3, \ell)\text{-DQDSD-PT}(\mathbf{w}_3, \mathbf{b}, \pi)$ problem for $\mathbf{w}_3 := (\mathbf{w}_r, \mathbf{w}_t, \mathbf{w}_e)$, such that

$$\text{Adv}_{\text{Mito}_\ell, \mathcal{A}}^{\text{IND-CPA}}(\lambda) \leq \text{Adv}_{(2, \ell)\text{-DQDSD-P}, \mathcal{B}_1}(\lambda) + \text{Adv}_{(3, \ell)\text{-DQDSD-PT}, \mathcal{B}_2}(\lambda).$$

We note that whether the parameters are *balanced* is not a fundamental concern. The difference between choosing an element of weight w from $\mathbb{F}_2^{pn}$ and choosing elements of weights w_i such that $\sum w_i = w$ from $\mathbb{F}_2^n$ p times is entirely dictated by the size of the probability space. In particular, for a space $\mathbb{F}_2^{pn}$ of length pn , the space *collapse* induced by the *disjoint partition weight* $\mathbf{w} = (w_1, w_2, \dots, w_p) \in \mathbb{N}^p$ is formulated as

$$\text{collapse}(\mathbf{w}) := \log_2 \left(\binom{pn}{\sum_{i=1}^p w_i} \right) - \sum_{i=1}^p \log_2 \left(\binom{n - \sum_{j \in \text{Dis}(i) \cap [i]} w_j}{w_i} \right)$$

where $\text{Dis}(i)$ is the set of indices that are disjoint from the support of the vector at index i . It is generally assumed that the asymptotic complexity of applying ISD algorithms to find a low-weight composite vector $\mathbf{e} \in (\mathbb{F}_2^n)^p$ with disjoint weight vector $\mathbf{w}(\mathbf{e}) = (w_1, \dots, w_p)$ satisfying the linear system

$$\mathbf{e} \cdot H^\top = \mathbf{s}$$

for $H \xleftarrow{\$} (\mathbb{F}_2^n)^{q \times p}$ is at least

$$O \left(2^{-\log_2 \left(\frac{q}{p} \right) (\sum_i w_i) - \text{collapse}(\mathbf{w})} \right).$$

Finally, given that the $(3, \ell)\text{-DQDSD-PT}$ instance constitutes a punctured code derived from $(3, \ell)\text{-DQDSD-P}$, ISD algorithms do not exhibit a non-polynomial factor advantage over this structure. Let $\mathbf{w}_\dagger = (w_\dagger[0], \dots, w_\dagger[2^\ell - 1])$ for $\dagger \in \{x, y, r, t\}$. For the ISD problem with unbalanced parameters, if asymptotically there exists no constant $c > 0$ such that

$$\max\{\text{collapse}(\mathbf{w}_x, \mathbf{w}_y), \text{collapse}(\mathbf{w}_r, w_e, \mathbf{w}_t)\} \geq cn,$$

then the security bounds of $\text{Mito}_\ell.\text{PKE}$ targeting a security level λ are characterized by the following two inequalities:

$$\lambda \leq \sum_i (w_x[i] + w_y[i]) - \text{collapse}(\mathbf{w}_x, \mathbf{w}_y), \quad (4.3)$$

$$\lambda \leq \log_2 \left(\frac{2^{\ell+1} + 1}{2^\ell + 1} \right) \left(w_e + \sum_i (w_r[i] + w_t[i]) \right) - \text{collapse}(\mathbf{w}_r, w_e, \mathbf{w}_t). \quad (4.4)$$

Let Trun denote the function that truncates the rightmost $(n - n')$ bits of an n -bit string, and (Encode, Decode) be the encoding and decoding procedures of the $[n', k]$ -linear code utilized by the scheme for error correction. To conclude, we present the pseudocode for Mito.PKE instantiated with $\ell = 1$ and $\ell = 2$.

4.2 Analysis of the Decoding Failure Rate

To ensure the correctness of the PKE and the security of the KEM, a rigorous upper bound on the DFR for Mito must be established. Following the derivation in [Definition 4.1.3](#), the decryption error of Mito.PKE originates from the equivalent error vector

$$\mathbf{e}_{\text{eq}} = \mathbf{e} + \sum_{i=0}^{2^\ell-1} (\mathbf{x}_i * \mathbf{t}_i - \mathbf{y}_i * \mathbf{r}_i),$$

where $*$ denotes polynomial multiplication in $\mathcal{R}_0 = \mathbb{F}_2[X]/\langle X^n - 1 \rangle$.

A decryption failure occurs if the Hamming weight of the equivalent error, $w(\mathbf{e}_{\text{eq}})$, exceeds the error-correcting capacity of the RSRM concatenated code. To estimate the distribution of $w(\mathbf{e}_{\text{eq}})$, we adopt the widely-used assumption from HQC, which models the weight distribution of polynomial products as a sum of independent Bernoulli trials. Specifically, we utilize the following proposition:

Proposition 4.2.1 ([\[GAMA⁺25, Proposition 6.1.1\]](#)). Let $\mathbf{x} = (x_0, \dots, x_{n-1})$ be a random vector chosen uniformly from all binary vectors of weight ω , and $\mathbf{r} = (r_0, \dots, r_{n-1})$ be a random vector of weight ω_r chosen uniformly and independently of $\mathbf{x}$. Let $\mathbf{z} = \mathbf{x} * \mathbf{r}$. Then, for any coordinate $k \in \{0, \dots, n-1\}$, the k -th entry z_k follows a Bernoulli distribution with parameter $\tilde{p} = \mathbb{P}[z_k = 1]$ given by

$$\tilde{p} := \tilde{p}(\omega, \omega_r) = \frac{1}{\binom{n}{\omega} \binom{n}{\omega_r}} \sum_{\substack{1 \leq \iota \leq \min(\omega, \omega_r) \\ \iota \text{ is odd}}} C_\iota,$$

where $C_\iota = \binom{n}{\iota} \binom{n-\iota}{\omega-\iota} \binom{n-\omega}{\omega_r-\iota}$ denotes the number of ways to choose the vectors such that their supports intersect at exactly ι positions.

Remark 4.2.1. We also note recent studies [\[BDM⁺25, MTV25, EA25\]](#) that have demonstrated, through both mathematical analysis and empirical simulations, that HQC tends to overestimate the weight of polynomial multiplications. Given that Mito introduces innovations in both algebraic structure and coding theory, we maintain a *conservative* approach by adhering to the classical weight estimation for polynomial products.

Since the entries of $\mathbf{e}_{\text{eq}}$ are independent, as justified in [Remark 4.2.1](#), let p_i denote the probability of a coordinate being non-zero in the i -th branch of the error sum, i.e.,

$$p_i = f(\tilde{p}(w_x[i], w_t[i]), \tilde{p}(w_y[i], w_r[i])).$$

The total coordinate probability $p^* = \mathbb{P}[\mathbf{e}_{\text{eq}}[k] = 1]$ is then given by

$$p^* = \begin{cases} f(p_0, p_1, w_e/n) & \text{for } \ell = 1, \\ f(p_0, p_1, p_2, p_3, w_e/n) & \text{for } \ell = 2, \end{cases}$$

where $f(p_1, \dots, p_m)$ denotes the probability that an *odd* number of independent Bernoulli events with parameters p_j occur. Specifically, $f(p_1, p_2) = p_1(1 - p_2) + p_2(1 - p_1)$.

Mito₁.KEM($\mathbf{w}_x, \mathbf{w}_y, \mathbf{w}_r, \mathbf{w}_t$)

KeyGen(1^λ) :

1. $(\mathbf{y}_0, \mathbf{y}_1) \xleftarrow{\$} \mathcal{S}^D(\mathbf{w}_y), (\mathbf{x}_0, \mathbf{x}_1) \xleftarrow{\$} \mathcal{S}^D(\mathbf{w}_x)$
 2. $\mathbf{h}_0, \mathbf{h}_1 \xleftarrow{\$} \mathbb{F}_2^n$
 3. $\mathbf{s}_0 \leftarrow \mathbf{x}_0 + \mathbf{y}_0 * \mathbf{h}_0 + \mathbf{y}_1 * \mathbf{h}_1,$
 $\mathbf{s}_1 \leftarrow \mathbf{x}_1 + \mathbf{y}_0 * \mathbf{h}_1 + \mathbf{y}_1 * \mathbf{h}_0$
 4. $\text{pk} \leftarrow (\mathbf{s}_0, \mathbf{s}_1, \mathbf{h}_0, \mathbf{h}_1), \text{sk} \leftarrow (\mathbf{y}_0, \mathbf{y}_1)$
 5. **return** (pk, sk)
-

Encrypt(pk, $\mathbf{m}, \theta$) :

1. $(\mathbf{r}_0, \mathbf{r}_1) \xleftarrow{\theta} \mathcal{S}^D(\mathbf{w}_r), (\mathbf{t}_0, \mathbf{t}_1) \xleftarrow{\theta} \mathcal{S}^D(\mathbf{w}_t)$
 2. $\mathbf{e} \xleftarrow{\theta} \mathcal{S}^D(\mathbf{w}_e)$
 3. $\mathbf{u}_0 \leftarrow \mathbf{r}_0 + \mathbf{t}_0 * \mathbf{h}_0 + \mathbf{t}_1 * \mathbf{h}_1,$
 $\mathbf{u}_1 \leftarrow \mathbf{r}_1 + \mathbf{t}_0 * \mathbf{h}_1 + \mathbf{t}_1 * \mathbf{h}_0$
 4. $\mathbf{v} \leftarrow \text{Encode}(\mathbf{m}) + \text{Trun}(\mathbf{e} + \mathbf{t}_0 * \mathbf{s}_0 + \mathbf{t}_1 * \mathbf{s}_1)$
 5. $\mathbf{c} \leftarrow (\mathbf{u}_0, \mathbf{u}_1, \mathbf{v})$
 6. **return** $\mathbf{c}$
-

Decrypt(sk, $\mathbf{c}$) :

1. $\mathbf{m} \leftarrow \text{Decode}(\mathbf{v} - \text{Trun}(\mathbf{y}_0 * \mathbf{u}_0 + \mathbf{y}_1 * \mathbf{u}_1))$
 2. **return** $\mathbf{m}$
-

Figure 4.4. The pseudocode for Mito₁.KEM

Mito₂.KEM($\mathbf{w}_x, \mathbf{w}_y, \mathbf{w}_r, \mathbf{w}_t$)

KeyGen(1^λ) :

1. $(y_0, y_1, y_2, y_3) \xleftarrow{\$} \mathcal{S}^D(\mathbf{w}_y), (x_0, x_1, x_2, x_3) \xleftarrow{\$} \mathcal{S}^D(\mathbf{w}_x)$
 2. $\mathbf{h}_0, \mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3 \xleftarrow{\$} \mathbb{F}_2^n$
 3. $\mathbf{s}_0 \leftarrow \mathbf{x}_0 + y_0 * \mathbf{h}_0 + y_1 * \mathbf{h}_1 + y_2 * \mathbf{h}_2 + y_3 * \mathbf{h}_3$
 $\mathbf{s}_1 \leftarrow \mathbf{x}_1 + y_0 * \mathbf{h}_1 + y_1 * \mathbf{h}_0 + y_2 * \mathbf{h}_3 + y_3 * \mathbf{h}_2$
 $\mathbf{s}_2 \leftarrow \mathbf{x}_2 + y_0 * \mathbf{h}_2 + y_1 * \mathbf{h}_3 + y_2 * \mathbf{h}_0 + y_3 * \mathbf{h}_1$
 $\mathbf{s}_3 \leftarrow \mathbf{x}_3 + y_0 * \mathbf{h}_3 + y_1 * \mathbf{h}_2 + y_2 * \mathbf{h}_1 + y_3 * \mathbf{h}_0$
 4. $\text{pk} \leftarrow (\mathbf{s}_0, \mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3, \mathbf{h}_0, \mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3), \text{sk} \leftarrow (y_0, y_1, y_2, y_3)$
 5. **return** (pk, sk)
-

Encrypt(pk, m, θ) :

1. $(\mathbf{r}_0, \mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3) \xleftarrow{\theta} \mathcal{S}^D(\mathbf{w}_r), (\mathbf{t}_0, \mathbf{t}_1, \mathbf{t}_2, \mathbf{t}_3) \xleftarrow{\theta} \mathcal{S}^D(\mathbf{w}_t)$
 2. $\mathbf{e} \xleftarrow{\theta} \mathcal{S}^D(\mathbf{w}_e)$
 3. $\mathbf{u}_0 \leftarrow \mathbf{r}_0 + \mathbf{t}_0 * \mathbf{h}_0 + \mathbf{t}_1 * \mathbf{h}_1 + \mathbf{t}_2 * \mathbf{h}_2 + \mathbf{t}_3 * \mathbf{h}_3$
 $\mathbf{u}_1 \leftarrow \mathbf{r}_1 + \mathbf{t}_0 * \mathbf{h}_1 + \mathbf{t}_1 * \mathbf{h}_0 + \mathbf{t}_2 * \mathbf{h}_3 + \mathbf{t}_3 * \mathbf{h}_2$
 $\mathbf{u}_2 \leftarrow \mathbf{r}_2 + \mathbf{t}_0 * \mathbf{h}_2 + \mathbf{t}_1 * \mathbf{h}_3 + \mathbf{t}_2 * \mathbf{h}_0 + \mathbf{t}_3 * \mathbf{h}_1$
 $\mathbf{u}_3 \leftarrow \mathbf{r}_3 + \mathbf{t}_0 * \mathbf{h}_3 + \mathbf{t}_1 * \mathbf{h}_2 + \mathbf{t}_2 * \mathbf{h}_1 + \mathbf{t}_3 * \mathbf{h}_0$
 4. $\mathbf{v} \leftarrow \text{Encode}(\mathbf{m}) + \text{Trun}(\mathbf{e} + \mathbf{t}_0 * \mathbf{s}_0 + \mathbf{t}_1 * \mathbf{s}_1 + \mathbf{t}_2 * \mathbf{s}_2 + \mathbf{t}_3 * \mathbf{s}_3)$
 5. $\mathbf{c} \leftarrow (\mathbf{u}_0, \mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3, \mathbf{v})$
 6. **return** c
-

Decrypt(sk, c) :

1. $\mathbf{m} \leftarrow \text{Decode}(\mathbf{v} - \text{Trun}(y_0 * \mathbf{u}_0 + y_1 * \mathbf{u}_1 + y_2 * \mathbf{u}_2 + y_3 * \mathbf{u}_3))$
 2. **return** m
-

Figure 4.5. The pseudocode for Mito₂.KEM

4.2.1 DFR Estimation for Classical RSRM Concatenated Codes

For the classical RSRM concatenated codes, we follow the methodology established in [GAMA⁺25] to evaluate the decoding performance. We first analyze the decoding failure probability of the internal duplicated RM code over a memoryless binary symmetric channel with a bit error rate p^* :

Proposition 4.2.2 (Improved Upper Bound for Internal DFR [GAMA⁺25, Proposition 6.1.4]). Let p^* be the transition probability of the binary symmetric channel. The decoding failure rate of the inner $[n_{\text{in}}, k_{\text{in}}, d_{\text{in}}]$ -code t -RM(1, m) can be upper bounded by

$$p_{\text{in}} = \sum_{\omega=d_{\text{in}}/2}^{n_{\text{in}}} A_{\omega} (p^*)^{\omega} (1-p^*)^{n_{\text{in}}-\omega},$$

where the coefficients

$$A_{\omega} := \min \left[\binom{n_{\text{in}}}{\omega}, \frac{1}{2} \cdot 255 \binom{d_{\text{in}}}{d_{\text{in}}/2} \binom{d_{\text{in}}}{\omega - d_{\text{in}}} + 255 \sum_{j=d_{\text{in}}/2+1}^{d_{\text{in}}} \binom{d_{\text{in}}}{j} \binom{d_{\text{in}}}{\omega - j} + \frac{1}{2} \binom{255}{2} \sum_{j=0}^{d_{\text{in}}/2} \binom{d_{\text{in}}/2}{j} \binom{d_{\text{in}}/2}{\omega - d_{\text{in}} + j} \right].$$

Next, we determine the total DFR by evaluating the decoding performance of the external code. The following theorem provides the upper bound for the concatenated system:

Theorem 4.2.1 (DFR of the RSRM concatenated code [GAMA⁺25, Theorem 6.1]). The DFR of the RSRM concatenated code, which utilizes a shorten RS code $[N_{\text{out}}, K_{\text{out}}, D_{\text{out}}]_{2^8}$ as the outer code and a duplicated RM code as the inner code, is upper bounded by

$$\text{DFR} \leq \sum_{l=\delta+1}^{N_{\text{out}}} \binom{N_{\text{out}}}{l} p_i^l (1-p_i)^{N_{\text{out}}-l},$$

where $D_{\text{out}} = 2\delta + 1$, and p_i is the decoding failure probability of the internal RM code as defined in Proposition 4.2.2.

In our parameter set, Mito₁ utilizes a traditional RSRM cascade. The estimated upper bounds for the decryption error rate of its corresponding security-level inner codes are provided in Table 4.1 below:

λ	p^*	$[n_{\text{in}}, k_{\text{in}}, d_{\text{in}}]$	$[N_{\text{out}}, K_{\text{out}}, D_{\text{out}}]_{2^8}$	$\log_2(p_i)$	$\log_2(\text{DFR})$
128	0.3161	[384, 8, 192]	[40, 16, 25] _{2⁸}	-15.91	-173.34
256	0.3335	[384, 8, 192]	[88, 32, 57] _{2⁸}	-12.06	-272.79
512	0.3671	[640, 8, 320]	[166, 64, 103] _{2⁸}	-12.71	-516.22

Table 4.1. Detailed DFR analysis for Mito₁ instance.

4.2.2 DFR Estimation Based on Distance-Informed Erasure Decoding

Since we introduced erasures in Algorithm 4, the decoding results of the RM code are no longer solely governed by the error rate p_i . Instead, we need to establish the following theorem to bound these probabilities.

Theorem 4.2.2 ([WTW⁺26, Theorem 1]). Let p^* be the transition probability of the binary symmetric channel and α be the decoding threshold of the $[n_{\text{in}}, k_{\text{in}}, d_{\text{in}}]$ -duplicated RM code. By treating certain erasures as errors, we can upper bound p_{error} and p_{erasure} as

$$p_{\text{error}} = \sum_{w=0}^{n_{\text{in}}} \min \left[\sum_{2j-d_{\text{in}} > \alpha} 255 \binom{d_{\text{in}}}{j} \binom{n_{\text{in}}-d_{\text{in}}}{w-j}, \binom{n_{\text{in}}}{w} \right] (p^*)^w (1-p^*)^{n_{\text{in}}-w} \quad (4.5)$$

and

$$p_{\text{erasure}} = \sum_{w=0}^{n_{\text{in}}} \min \left[\sum_{-\alpha \leq d_{\text{in}}-2j \leq \alpha} 255 \binom{d_{\text{in}}}{j} \binom{n_{\text{in}}-d_{\text{in}}}{w-j}, \binom{n_{\text{in}}}{w} \right] (p^*)^w (1-p^*)^{n_{\text{in}}-w}. \quad (4.6)$$

Furthermore, let σ denote the number of errors and t denote the number of erasures. The decoding succeeds as long as $2\sigma + t \leq N_{\text{out}} - K_{\text{out}}$. Consequently, we obtain the estimate of the DFR as presented in [Theorem 4.2.3](#).

Theorem 4.2.3 ([WTW⁺26, Theorem 2]). The DFR of the outer $[N_{\text{out}}, K_{\text{out}}, D_{\text{out}}]$ -RS code is given by

$$\text{DFR} \leq \sum_{2\sigma+t > N_{\text{out}}-K_{\text{out}}} \binom{N_{\text{out}}}{\sigma} \binom{N_{\text{out}}-\sigma}{t} p_{\text{error}}^{\sigma} p_{\text{erasure}}^t (1-p_{\text{error}}-p_{\text{erasure}})^{N_{\text{out}}-\sigma-t}. \quad (4.7)$$

Within our parameter sets, Mito₁-E and Mito₂-E utilize the Distance-Informed Erasure Decoding technique. The estimated upper bounds for the DFR of their corresponding inner codes at respective security levels are detailed in [Table 4.2](#) and [Table 4.3](#) below:

λ	p^*	$[n_{\text{in}}, k_{\text{in}}, d_{\text{in}}]$	$[N_{\text{out}}, K_{\text{out}}, D_{\text{out}}]_{2^8}$	α	$\log_2(p_{\text{error}})$	$\log_2(p_{\text{erasure}})$	$\log_2(\text{DFR})$
128	0.3256	[384, 8, 192]	[38, 16, 23] _{2⁸}	9	-18.95	-9.49	-167.99
256	0.3420	[384, 8, 192]	[84, 32, 53] _{2⁸}	7	-13.94	-7.51	-265.48
512	0.3735	[640, 8, 320]	[160, 64, 97] _{2⁸}	11	-15.51	-7.46	-517.00

Table 4.2. Detailed DFR analysis for Mito₁-E instance.

λ	p^*	$[n_{\text{in}}, k_{\text{in}}, d_{\text{in}}]$	$[N_{\text{out}}, K_{\text{out}}, D_{\text{out}}]_{2^8}$	α	$\log_2(p_{\text{error}})$	$\log_2(p_{\text{erasure}})$	$\log_2(\text{DFR})$
128	0.2907	[256, 8, 128]	[40, 16, 25] _{2⁸}	7	-18.54	-9.55	-181.39
256	0.3079	[256, 8, 128]	[84, 32, 53] _{2⁸}	7	-15.03	-7.07	-269.97
512	0.3409	[384, 8, 192]	[170, 64, 107] _{2⁸}	7	-14.19	-7.69	-547.37

Table 4.3. Detailed DFR analysis for Mito₂-E instance.

4.3 Security Proof

4.3.1 Known Attacks Against the IND-CPA Security

Recall from [Corollary 4.1.1](#) that the IND-CPA security of Mito.PKE is reduced to the distinguishing advantage against the $(2, \ell)$ -QSD-P and $(3, \ell)$ -QSD-PT problems. Following [Theorem 4.1.2](#), the most efficient attacks against these two problems are the ISD algorithms applied to instances of the corresponding dimensions. Hence, the theoretical complexity of these two problems is established by (4.3) and (4.4).

Utilizing the estimation framework from [EVZB24], we summarize the time and memory complexities of mainstream ISD variants [Pra62, Ste88, BJMM12, BM17] for the specific SD instances across all security levels in Table 4.4, Table 4.5 and Table 4.6. Specifically, the practical IND-CPA security of Mito.PKE is lower-bounded by the minimum complexity of solving these underlying problems, i.e.,

$$\mathcal{O}(\text{Mito.PKE}^{\text{IND-CPA}}) \geq \min \{ \mathcal{O}((2, \ell)\text{-QSD}), \mathcal{O}((3, \ell)\text{-QSD}) \}.$$

The *quantum-safe security* is obtained by dividing the security bits by two (taking the square root of the complexity) [Ber10].

Table 4.4. ISD complexity estimates (Time / Memory in $\log_2$) for Mito₁

λ	Problem	Parameters			Algorithm Estimates ($\log_2$)			
		n_{SD}	k_{SD}	w_{SD}	Prange	Stern	BJMM	BM
128	(2, 1)-QSD	61492	30746	172	215.7 / 31.0	193.2 / 43.7	193.5 / 41.7	193.4 / 41.7
	(3, 1)-QSD	76865	30746	261	237.3 / 32.0	214.3 / 44.0	214.6 / 42.1	214.4 / 42.0
256	(2, 1)-QSD	135308	67654	269	316.0 / 33.3	291.0 / 47.1	291.2 / 45.1	291.1 / 45.1
	(3, 1)-QSD	169135	67654	407	348.2 / 34.2	322.7 / 47.5	322.9 / 45.5	322.7 / 64.6
512	(2, 1)-QSD	425044	212522	527	578.9 / 36.5	550.1 / 52.1	550.2 / 50.1	549.9 / 71.8
	(3, 1)-QSD	531305	212522	794	638.3 / 37.4	609.0 / 52.4	608.9 / 75.0	608.4 / 72.1

Table 4.5. ISD complexity estimates (Time / Memory in $\log_2$) for Mito₁-E

λ	Problem	Parameters			Algorithm Estimates ($\log_2$)			
		n_{SD}	k_{SD}	w_{SD}	Prange	Stern	BJMM	BM
128	(2, 1)-QSD	58484	29242	172	215.5 / 30.9	193.0 / 43.5	193.3 / 41.5	193.2 / 41.5
	(3, 1)-QSD	73105	29242	262	237.8 / 31.7	214.8 / 43.8	215.1 / 41.8	214.9 / 41.8
256	(2, 1)-QSD	129024	64512	269	315.8 / 33.1	290.8 / 46.9	291.0 / 44.9	291.0 / 44.9
	(3, 1)-QSD	161280	64512	406	347.3 / 34.0	321.8 / 47.3	321.8 / 66.0	321.8 / 67.0
512	(2, 1)-QSD	409748	204874	527	578.7 / 36.4	549.9 / 51.9	550.1 / 74.4	549.7 / 73.4
	(3, 1)-QSD	512185	204874	793	637.4 / 37.3	608.1 / 52.3	607.9 / 74.7	607.5 / 73.7

Table 4.6. ISD complexity estimates (Time / Memory in $\log_2$) for Mito₂-E

λ	Problem	Parameters			Algorithm Estimates ($\log_2$)			
		n_{SD}	k_{SD}	w_{SD}	Prange	Stern	BJMM	BM
128	(2, 2)-QSD	82024	41012	186	230.8 / 31.8	207.9 / 45.0	208.2 / 43.0	208.1 / 43.0
	(3, 2)-QSD	92277	41012	229	239.7 / 32.4	216.6 / 45.1	216.8 / 43.1	216.8 / 43.1
256	(2, 2)-QSD	172184	86092	284	332.0 / 33.9	306.7 / 48.2	306.8 / 46.2	306.9 / 46.2
	(3, 2)-QSD	193707	86092	356	350.5 / 34.5	325.0 / 48.4	325.1 / 46.4	325.1 / 46.4
512	(2, 2)-QSD	522344	261172	544	596.7 / 37.2	567.7 / 53.0	567.8 / 51.0	567.9 / 51.0
	(3, 2)-QSD	587637	261172	683	632.5 / 37.6	603.3 / 53.2	603.4 / 76.0	603.4 / 72.1

DOOM Attack. For quasi-cyclic schemes, the DOOM attack [Sen11] provides a speedup factor of $\mathcal{O}(\sqrt{n})$ for ISD algorithms by constructing a collision list. This is because quasi-cyclic codes fall

under the category of reproducible codes, and their transform set $T_{\mathcal{C}} = \{L_X^k \mid k \in [n]\}$ has a size of exactly n .

More generally, considering the action of the group $\langle X, T_1, \dots, T_\ell \mid X^n = 1, T_i^2 = 1 \rangle$ on the ring $\mathbb{R}_\ell$, we effectively obtain a collision list of size $2^\ell n$. Consequently, the DOOM speedup factor for quasi-dyadic structured codes scales to $\mathcal{O}(\sqrt{2^\ell n})$. Combining this with our baseline ISD estimations, we present the practical security complexities adjusted for the DOOM acceleration in [Table 4.7](#).

Table 4.7. Practical ISD complexity estimates factoring in the DOOM attack

Scheme	λ	DOOM Factor ($\log_2$)	DOOM-ISD Complexity (Time / Mem.)
Mito ₁	128	7.45	185.8 / 43.7
	256	8.02	283.0 / 47.1
	512	8.85	541.1 / 71.8
Mito ₁ -E	128	7.42	185.6 / 43.5
	256	7.99	282.8 / 46.9
	512	8.82	540.9 / 73.4
Mito ₂ -E	128	7.66	200.2 / 45.0
	256	8.20	298.5 / 48.2
	512	9.00	558.7 / 53.0

Remark 4.3.1. We note that our theoretical bound $\mathcal{O}(2^\lambda)$ provides a conservative security margin. The actual concrete complexity evaluated by ISD estimators is typically around 30 bits higher, primarily because the asymptotic bound omits the substantial polynomial overhead incurred in each iteration of the ISD algorithms.

Square Attack. Another attack worth mentioning originates from [\[LJKS⁺16\]](#), which targets algebraic structures of the form

$$\mathbb{R}_\ell := \mathbb{F}_2[X] / \langle X^{2^\ell n} + 1 \rangle.$$

As a ring or $\mathbb{F}_2$ -algebra, $\mathbb{R}_\ell$ is not isomorphic to $\mathcal{R}_\ell$. The image of the squaring map $\mathbf{r} \mapsto \mathbf{r}^2$ acting on $\mathcal{R}_\ell$ is actually $\mathcal{R}_0$. In this case, the equation $\mathbf{x} + \mathbf{y} * \mathbf{h} = \mathbf{s}$ over $\mathcal{R}_\ell$ transforms into

$$\mathbf{x}_{\text{sum}} + \mathbf{y}_{\text{sum}} * \mathbf{h}_{\text{sum}} = \mathbf{s}_{\text{sum}}$$

over $\mathcal{R}_0$, where for $\mathbf{r} = (r_0, \dots, r_{2^\ell-1}) \in \mathcal{R}_\ell$, we have

$$\mathbf{r}_{\text{sum}} := \sum_{i=0}^{2^\ell-1} \mathbf{r}_i.$$

Since Mito uses elements from $\mathcal{S}^D(\mathbf{w})$ as error vectors, the aforementioned transformation does not change the weight value as an ISD instance, but rather reduces solving the QDSD problem to solving the QCSD problem. Therefore, this attack does not introduce a super-polynomial security loss for Mito.

4.3.2 IND-CCA Security of KEM

Since we follow the widely adopted paradigm of constructing a KEM from a PKE scheme via the FO transform, the IND-CCA security of our KEM is determined by the IND-CPA security and the DFR of the underlying PKE. Generally, we have the following definitions:

Definition 4.3.1 (δ -Correctness for PKE). A PKE scheme $\Pi = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt})$ with message space $\mathcal{M}$ is called δ -correct if

$$\mathbb{E} \left[\max_{\mathbf{m} \in \mathcal{M}} \mathbb{P}_{\theta \leftarrow \mathcal{R}} [\text{Decrypt}(\text{dk}, \text{Encrypt}(\text{ek}, \mathbf{m}, \theta)) \neq \mathbf{m}] \right] \leq \delta,$$

where the expectation is taken over the randomness of KeyGen, and the inner probability is taken over the uniform choice of randomness $\theta \in \mathcal{R}$ used by Encrypt.

Definition 4.3.2 (δ -Correctness for KEM). A KEM scheme $\Gamma = (\text{KeyGen}, \text{Encaps}, \text{Decaps})$ is called δ -correct if

$$\mathbb{P} \left[\text{Decaps}(\text{dk}, \text{ct}) \neq K \mid \begin{array}{l} (\text{ek}, \text{dk}) \leftarrow \text{KeyGen}(1^\lambda), \\ r \leftarrow \mathcal{R}, \\ (K, \text{ct}) \leftarrow \text{Encaps}(\text{ek}, r) \end{array} \right] \leq \delta,$$

where the probability is taken over the internal randomness of KeyGen and the explicit randomness r used by Encaps.

The decryption error of Mito.PKE stems exclusively from the Hamming weight of the equivalent error $\mathbf{e}_{\text{eq}}$ exceeding the decoding capacity of the concatenated code. Furthermore, $\mathbf{e}_{\text{eq}}$ is completely independent of the encrypted message $\mathbf{m}$. Therefore, the maximum probability over all messages in [Definition 4.3.1](#) simplifies to

$$\mathbb{P}[\text{Mito.PKE.Decrypt}(\text{dk}, \text{ct}) \neq \mathbf{m} \mid \text{ct} \leftarrow \text{Mito.PKE.Encrypt}(\text{ek}, \mathbf{m})] \leq \delta, \quad (4.8)$$

which is exactly equivalent to the DFR analyzed in [Section 4.2](#).

The following theorem demonstrates that Mito.KEM achieves IND-CCA security in the Random Oracle Model. Its concrete advantage bound is obtained by combining the SFO transformation security estimates [[GHS25](#), Theorem 12] with our IND-CPA reduction established in [Corollary 4.1.1](#).

Theorem 4.3.1 (IND-CCA Security of $\text{Mito}_\ell.\text{KEM}$). Assuming $\text{Mito}_\ell.\text{PKE}$ is δ -correct, for any IND-CCA adversary $\mathcal{A}$ against $\text{Mito}_\ell.\text{KEM}$ issuing at most q_{RO} queries to the random oracle $\mathcal{G}$ and q_D queries to the decapsulation oracle, there exist adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that

$$\begin{aligned} \text{Adv}_{\text{Mito}_\ell.\text{KEM}, \mathcal{A}}^{\text{IND-CCA}}(\lambda) &\leq \frac{1}{2^{\lambda_{\text{salt}}} \cdot 2^{8k}} + \frac{3q_{\text{RO}}}{2^{8k}} + (q_{\text{RO}} + q_D) \cdot \delta \\ &\quad + 2 \cdot (\text{Adv}_{(2,\ell)\text{-DQSD-P}, \mathcal{B}_1}(\lambda) + \text{Adv}_{(3,\ell)\text{-DQSD-PT}, \mathcal{B}_2}(\lambda)). \end{aligned} \quad (4.9)$$

In the quantum setting, considering the absence of tight proofs in the Quantum Random Oracle Model (QROM) for KEMs derived via the FO transformation, we adopt a conservative approach for the quantum security level λ_Q . Specifically, we estimate the KEM advantage in the quantum sense by halving both the classical IND-CPA advantage and the DFR of the underlying PKE.

4.3.3 Advantages and Limitations

We summarize the primary advantages and the inherent trade-offs of the proposed Mito KEM framework.

Advantages.

- **Theoretical Rigor and Hard Problem Innovation:** We introduce a generalized algebraic framework that rigorously formalizes the security assumptions of both quasi-cyclic and quasi-dyadic schemes. This abstraction not only elucidates the security of existing schemes like HQC but also establishes a solid theoretical foundation for the Quasi-Dyadic Syndrome Decoding (QDSD) problem.

- **Versatile Parameterization for Diverse Scenarios:** By introducing the adic number ℓ , our scheme can be flexibly tailored to accommodate diverse resource-constrained scenarios.
- **Reduced Ciphertext Size and Accelerated Decryption:** Leveraging the inherent block-wise properties of the dyadic structure, Mito significantly mitigates the equivalent error weight processed during decoding. This structural advantage substantially accelerates the decryption algorithm. Furthermore, it enables the integration of underlying codes with stronger error-correcting capabilities, ultimately yielding a more compact ciphertext compared to state-of-the-art schemes like HQC.

Limitations.

- **Larger Public Key Footprint:** The primary trade-off of the quasi-dyadic construction is a larger public key size and increased key generation time compared to HQC. Consequently, Mito is optimally suited for network scenarios where public keys are static and cached, rather than ephemeral key-exchange settings where bandwidth is heavily constrained.
- **Security Gap Between the Asymptotic and Practical ISD Complexity:** Partitioning the bit string into more blocks introduces a more pronounced gap between the asymptotic theoretical security bounds and the practical complexity evaluated by ISD variants. This makes our scheme more conservative.

Bibliography

- [AMAD⁺24] Carlos Aguilar-Melchor, Nicolas Aragon, Jean-Christophe Deneuville, Philippe Gaborit, Jérôme Lacan, and Gilles Zémor. Efficient error-correcting codes for the hqc post-quantum cryptosystem. *Designs, Codes and Cryptography*, 92(12):4511–4530, 2024.
- [AMBD⁺18] Carlos Aguilar-Melchor, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, and Gilles Zémor. Efficient encryption from random quasi-cyclic codes. *IEEE Transactions on Information Theory*, 64(5):3927–3943, 2018.
- [ASY20] Emmanuel Abbe, Amir Shpilka, and Min Ye. Reed–muller codes: Theory and algorithms. *IEEE Transactions on Information Theory*, 67(6):3251–3277, 2020.
- [BBB⁺18] Gustavo Banegas, Paulo SLM Barreto, Brice Odilon Boidje, Pierre-Louis Cayrel, Gilbert Ndollane Dione, Kris Gaj, Cheikh Thiécoumba Gueye, Richard Haeussler, Jean Belo Klamti, Ousmane N’diaye, et al. Dags: Key encapsulation using dyadic gs codes. *Journal of Mathematical Cryptology*, 12(4):221–239, 2018.
- [BBC08] Marco Baldi, Marco Bodrato, and Franco Chiaraluce. A new analysis of the McEliece cryptosystem based on QC-LDPC codes. In Rafail Ostrovsky, Roberto De Prisco, and Ivan Visconti, editors, *SCN 08*, volume 5229 of *LNCS*, pages 246–262. Springer, Berlin, Heidelberg, September 2008.
- [BBC⁺19] Marco Baldi, Alessandro Barengi, Franco Chiaraluce, Gerardo Pelosi, and Paolo Santini. Ledacrypt: Qc-ldpc code-based cryptosystems with bounded decryption failure rate. In *Code-Based Cryptography Workshop*, pages 11–43. Springer, 2019.
- [BC18] Élise Barelli and Alain Couvreur. An efficient structural attack on nist submission dags. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 93–118. Springer, 2018.
- [BCGO09] Thierry P. Berger, Pierre-Louis Cayrel, Philippe Gaborit, and Ayoub Otmani. Reducing key length of the McEliece cryptosystem. In Bart Preneel, editor, *AFRICACRYPT 09*, volume 5580 of *LNCS*, pages 77–97. Springer, Berlin, Heidelberg, June 2009.
- [BDM⁺25] Sebastian Bitzer, Jean-Christophe Deneuville, Emma Munisamy, Bharath Purtipli, Stefan Ritterhoff, and Antonia Wachter-Zeh. Hqc beyond the standard: Ciphertext compression and refined dfr analysis. *Cryptology ePrint Archive*, 2025.
- [Ber10] Daniel J Bernstein. Grover vs. mceliece. In *International Workshop on Post-Quantum Cryptography*, pages 73–80. Springer, 2010.
- [BJMM12] Anja Becker, Antoine Joux, Alexander May, and Alexander Meurer. Decoding random binary linear codes in $2^{n/20}$: How $1 + 1 = 0$ improves information set decoding. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 520–536. Springer, 2012.
- [BM17] Leif Both and Alexander May. Optimizing bjmm with nearest neighbors: full decoding in $2^{22/21n}$ and mceliece security. In *WCC workshop on coding and cryptography*, volume 214, 2017.
- [BS86] Yair Be’ery and Jakov Snyders. Optimal soft decision block decoders based on fast hadamard transform. *IEEE transactions on information theory*, 32(3):355–364, 1986.

- [BZ73] È L Blokh and Victor Vasilievich Zyablov. Existence of linear concatenated binary codes with optimal correcting properties. *Problemy Peredachi Informatsii*, 9(4):3–10, 1973.
- [Dum86] II Dumer. On syndrome decoding of linear codes. In *Proc. Ninth All-Union Symp. Redundancy in Information Systems*. Nauka, volume 2, pages 157–159, 1986.
- [EA25] Laila El Aimani. Weight of polynomial products mod $(x^n + 1)$ -application to the hqc cryptosystem. 2025.
- [EVZB24] Andre Esser, Javier A. Verbel, Floyd Zweydinger, and Emanuele Bellini. {SoK: CryptographicEstimators - a Software Library for Cryptographic Hardness Estimation}. In *AsiaCCS*. {ACM}, 2024.
- [FOP⁺16] Jean-Charles Faugère, Ayoub Otmani, Ludovic Perret, Frédéric de Portzamparc, and Jean-Pierre Tillich. Structural cryptanalysis of McEliece schemes with compact keys. *DCC*, 79(1):87–112, 2016.
- [FOPT10] Jean-Charles Faugère, Ayoub Otmani, Ludovic Perret, and Jean-Pierre Tillich. Algebraic cryptanalysis of McEliece variants with compact keys. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 279–298. Springer, Berlin, Heidelberg, May / June 2010.
- [Gab05] Philippe Gaborit. Shorter keys for code based cryptography. In *Proceedings of the 2005 International Workshop on Coding and Cryptography (WCC 2005)*, pages 81–91, 2005.
- [GAMA⁺25] Philippe Gaborit, Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, et al. Hamming quasi-cyclic (hqc). *NIST Post-Quantum Standardization, 4th Round; National Institute of Standards and Technology: Gaithersburg, MD, USA*, 2025.
- [GHS25] Lewis Glabush, Kathrin Hövelmanns, and Douglas Stebila. Tight multi-challenge security reductions for key encapsulation mechanisms. *IACR Cryptol. ePrint Arch.*, 2025:343, 2025.
- [GJL15] Qian Guo, Thomas Johansson, and Carl Löndahl. A new algorithm for solving ring-lpn with a reducible polynomial. *IEEE Transactions on Information Theory*, 61(11):6204–6212, 2015.
- [Gre66] Richard R Green. A serial orthogonal decoder. *JPL Space Programs Summary*, 37:247–253, 1966.
- [HHK17] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 341–371. Springer, Cham, November 2017.
- [LJKS⁺16] Carl Löndahl, Thomas Johansson, Masoumeh Koochak Shooshtari, Mahmoud Ahmadian-Attari, and Mohammad Reza Aref. Squaring attacks on mceliece public-key cryptosystems using quasi-cyclic codes of even dimension. *Designs, Codes and Cryptography*, 80(2):359–377, 2016.
- [McE78] Robert J. McEliece. A public-key cryptosystem based on algebraic coding theory. The deep space network progress report 42-44, Jet Propulsion Laboratory, California Institute of Technology, January/February 1978. https://ipnpr.jpl.nasa.gov/progress_report2/42-44/44N.PDF.

- [MK22] Meraiah Martinez and Christine A Kelley. Minimum distance and other properties of quasi-dyadic parity check codes. In *2022 IEEE International Symposium on Information Theory (ISIT)*, pages 2118–2123. IEEE, 2022.
- [MMT11] Alexander May, Alexander Meurer, and Enrico Thomae. Decoding random linear codes in. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 107–124. Springer, 2011.
- [MO15] Alexander May and Ilya Ozerov. On computing nearest neighbors with applications to decoding of binary linear codes. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 203–228. Springer, 2015.
- [MPK25] Meraiah Martinez, Tefjol Pllaha, and Christine A Kelley. Codes based on dyadic matrices and their generalizations. *Advances in Mathematics of Communications*, 19(5):1277–1300, 2025.
- [MTV25] Antoine Mesnard, Jean-Pierre Tillich, and Valentin Vasseur. The syndrome weight distribution in quasi-cyclic codes, applications to bike and hqc. *Cryptology ePrint Archive*, 2025.
- [OTD10] Ayoub Otmani, Jean-Pierre Tillich, and Léonard Dallot. Cryptanalysis of two mceliece cryptosystems based on quasi-cyclic codes. *Math. Comput. Sci.*, 3(2):129–140, 2010.
- [Per12] Edoardo Persichetti. Compact mceliece keys based on quasi-dyadic srivastava codes. *J. Math. Cryptol.*, 6(2):149–169, 2012.
- [Pra62] Eugene Prange. The use of information sets in decoding cyclic codes. *IRE Transactions on Information Theory*, 8(5):5–9, 1962.
- [Sen11] Nicolas Sendrier. Decoding one out of many. In *International Workshop on Post-Quantum Cryptography*, pages 51–67. Springer, 2011.
- [SPB21] Paolo Santini, Edoardo Persichetti, and Marco Baldi. Reproducible families of codes and cryptographic applications. *Journal of Mathematical Cryptology*, 16(1):20–48, 2021.
- [Ste88] Jacques Stern. A method for finding codewords of small weight. In *International colloquium on coding theory and applications*, pages 106–113. Springer, 1988.
- [Tho03] Christian Thommesen. The existence of binary linear concatenated codes with reed-solomon outer codes which asymptotically meet the gilbert-varshamov bound. *IEEE transactions on information theory*, 29(6):850–853, 2003.
- [WGR22] Violetta Weger, Niklas Gassner, and Joachim Rosenthal. A survey on code-based cryptography. *arXiv preprint arXiv:2201.07119*, 2022.
- [WTW⁺26] Tianrui Wang, Qicheng Teng, Anyu Wang, Jun Zhang, Bo Pang, Chunhuan Zhao, Sihuang Hu, and Xiaoyun Wang. Hare: Compact hqc via distance-informed erasure decoding. *Cryptology ePrint Archive*, 2026.